AWS Certified Cloud Practitioner

Manual de estudo — resumos das 14 aulas

CLF-C02 • Documento consolidado para impressão A4

Preparado a partir dos ficheiros Markdown fornecidos.

Índice

Aula 1 — Aula 1 — Fundamentos de Cloud Computing e AWS Cloud Practitioner
Aula 2 — Aula 2 — Infraestrutura Global da AWS e Conceitos de Negócio
Aula 3 — Aula 3 — AWS para Setor Público, Sustentabilidade e Arquitetura de Alta Disponibilidade
Aula 4 — Aula 4 — Ferramentas AWS, IaC e Shared Responsibility Model
Aula 5 — Aula 5 — Computing Services
Aula 6 — Aula 6 — Storage e Databases
Aula 7 — Aula 7 — Networking e EC2 em Detalhe
Aula 8 — Aula 8 — Auto Scaling, Pricing Models e Segurança de Identidade
Aula 9 — Aula 9 — IAM Avançado, Application Integration, Containers e Governança
Aula 10 — Aula 10
Aula 11 — Aula 11 — PaaS, Serverless, Windows, Logging, ML/AI e Big Data
Aula 12 — Aula 12 — Frameworks de ML, Hardware, Well-Architected Framework e Migração
Aula 13 — Aula 13 — Serviços Gratuitos, Planos de Suporte, Billing, Trusted Advisor, SLA e Custos
Aula 14 — Aula 14 — Pricing API, Segurança, Encriptação, Compliance e Serviços de Segurança AWS

Aula 1

Aula 1 — Fundamentos de Cloud Computing e AWS Cloud Practitioner

Aula 1 — Fundamentos de Cloud Computing e AWS Cloud Practitioner

■ Sobre a certificação

AWS Certified Cloud Practitioner (CLF-C02) — certificação de nível de entrada.

Roadmap sugerido para DevOps:

```
Cloud Practitioner → Solutions Architect Associate → CloudOps Engineer Associate → DevOps Engineer
```

Formato do exame

Aspeto	Detalhe
Duração	90 minutos
Nº de perguntas	65 (50 contam para a nota, 15 são de teste/não contam)
Tipo de perguntas	Escolha múltipla e resposta múltipla
Pontuação para passar	700 em 1000
Validade da certificação	3 anos

Domínios do exame

Domínio	Peso
1. Cloud Concepts	24%
2. Security and Compliance	30%
3. Cloud Technology and Services	34%
4. Billing, Pricing and Support	12%

Como estudar

- Ver vídeos das aulas
- Fazer hands-on labs
- Fazer exames de prática online

■■ O que é Cloud Computing

“Prática de usar uma rede de servidores remotos alojados na internet para armazenar, gerir e processar dados, em vez de usar um servidor local.”

Evolução do alojamento (hosting)

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["Dedicated Server<br/>1 máquina por negócio"] --> B["VPS<br/>1 máquina, virtualizada<br/>em sub-máquinas"]
    B --> C["Shared Hosting<br/>1 máquina física partilhada<br/>por centenas de negócios"]
    C --> D["Cloud Hosting<br/>Múltiplas máquinas físicas<br/>a agir como um único sistema"]
```

O que é um Cloud Service Provider (CSP)

Empresa que fornece múltiplos serviços cloud. Características típicas:

- Os serviços podem ser encadeados entre si
- Acessíveis através de uma API unificada
- Faturação metered (pagas pelo que consomes)
- Monitorização integrada
- Oferecem IaaS (Infrastructure as a Service)
- Automação via IaC (Infrastructure as Code)

Principais fornecedores: AWS, Azure, GCP

Categorias de serviços cloud

- Compute
- Networking
- Storage
- Databases

“A AWS tem mais de 200 serviços disponíveis.”

■ Modelos de serviço cloud

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph SaaS["SaaS"]
        S1["Aplicação pronta a usar.<br/>Não te preocupas com manutenção.<br/>Ex: Gmail, Office 365"]
    end
    subgraph PaaS["PaaS"]
        P1["Focas-te no deployment e gestão<br/>das tuas próprias aplicações.<br/>A infraestrutura é gerida pelo provider"]
    end
    subgraph IaaS["IaaS"]
        I1["Controlo total sobre servidores,<br/>rede e storage virtualizados.<br/>Ex: EC2, VPC"]
    end
```

Modelo	O que geres tu	O que é gerido pelo provider
IaaS	SO, runtime, apps, dados	Servidores físicos, virtualização, rede
PaaS	Apps e dados	SO, runtime, servidores, infraestrutura
SaaS	Apenas a utilização	Tudo (a app já vem pronta a usar)

■ Modelos de deployment

- Public cloud — infraestrutura partilhada, gerida por um provider (ex.: AWS)
- Private cloud — infraestrutura dedicada a uma única organização
- Hybrid cloud — combinação de cloud pública e privada
- Cross/Multi cloud — utilização de múltiplos providers cloud em simultâneo

■ IAM — primeiros passos

- Criar utilizadores e grupos no IAM (ex.: grupo admin com Administrator Access)
- Nunca usar a conta root para tarefas do dia a dia — usar sempre utilizadores IAM

■ Regiões

Uma região é uma área geográfica onde a AWS tem infraestrutura. Os recursos são criados dentro de uma região específica.

■ Billing e controlo de custos

“O custo é sempre proporcional ao que consomes: poder computacional × tempo em execução.”

Uma "história de overbilling" comum: recursos deixados a correr sem necessidade acabam por gerar custos inesperados — daí a importância de configurar alertas.

AWS Budgets / Billing Alerts

Fluxo típico para criar um alerta de custos:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A[CloudWatch] --> B[Billing]
    B --> C[Criar um Alarm]
    C --> D{Condição atingida?}
    D -->|Valor de budget ultrapassado| E[Notificação]
    D -->|Anomalia de custo detetada| E
    E --> F["Ação automática opcional<br/>(ex: desligar um serviço)"]
```

Pode-se definir o alarme para dois tipos de situação:

1. Um valor de budget específico foi atingido.
2. Algo anómalo aconteceu (ex.: aparece um custo excessivo sem explicação).

■ Progresso: 1h08min de vídeo assistido.

Aula 2

Aula 2 — Infraestrutura Global da AWS e Conceitos de Negócio

Aula 2 — Infraestrutura Global da AWS e Conceitos de Negócio

■ Configurar MFA (Multi-Factor Authentication)

Como ativar no root:

1. IAM → Add MFA
2. Instalar uma aplicação autenticadora (ex.: Google Authenticator, Authy)
3. Ler o QR code
4. Inserir dois códigos consecutivos gerados pela app

“■■ Recomendação de boas práticas: ativa sempre MFA na conta root e também nos utilizadores IAM com permissões elevadas.”

■ Contexto de transformação digital

(Conceitos de negócio/gestão — não são terminologia oficial da AWS, mas podem surgir como contexto conceptual.)

Ondas de inovação

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    P[Prosperity] --> R[Recession]
    R --> D[Depression]
    D --> E[Improvement]
    E --> P
```

Burning platform

“Conceito em que uma empresa abandona tecnologia antiga a favor de tecnologia nova, apesar da incerteza quanto ao sucesso — muitas vezes motivada pelo medo de que a sobrevivência futura da organização dependa da sua transformação digital.”

■ Evolução do poder computacional

Medido pela velocidade a que um computador consegue completar uma tarefa computacional.

Tipo de computação	Desempenho relativo	Serviço AWS associado
CPU (general computing)	Base	EC2
GPU computing	~50x mais rápido que CPU	EC2 (instâncias GPU), Inf1
Quantum computing	~100 milhões de vezes mais rápido	Amazon Braket

“Nota: Amazon Braket é um serviço secundário para o exame — não é um tema prioritário.”

■ Vantagens da cloud

“■■ Esta lista tem 7 pontos, mas o material oficial da AWS costuma apresentar 6 vantagens (trade CAPEX por OPEX, economias de escala, parar de adivinhar capacidade, aumentar velocidade e agilidade, parar de gerir data centers, ir global em minutos). Vale a pena confirmar se esta é uma variante do mesmo conteúdo dada pelo teu curso.”

Vantagem	Descrição
Cost effective	Pagas apenas pelo que consomes, sem custo inicial
Global	Podes lançar workloads em qualquer parte do mundo
Secure	O provider cuida da segurança física; os serviços podem ser seguros por defeito
Reliable	Backup de dados, disaster recovery, replicação, tolerância a falhas
Scalable	Aumentas ou reduzes recursos conforme a procura
Elastic	O scaling pode ser automatizado
Current	O hardware e software subjacentes são atualizados pelo provider sem interrupção

■ Infraestrutura Global da AWS

“Conjunto de hardware e data centers fisicamente interligados, distribuídos globalmente.”

Números atuais: 39 regiões · 123 availability zones · 135+ pontos de ligação direta · 700+ pontos de presença

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    G[AWS Global Infrastructure] --> R1[Região us-east-1]
    G --> R2[Região eu-west-1]
    G --> R3[...]
    R1 --> AZ1[AZ us-east-1a]
    R1 --> AZ2[AZ us-east-1b]
    R1 --> AZ3[AZ us-east-1c]
    AZ1 --> DC1[Data Center]
    AZ1 --> DC2[Data Center]
```

Regiões

- Localizações geograficamente distintas
- Cada região é fisicamente isolada e independente das restantes (localização, energia, água)
- O custo dos serviços AWS varia por região
- Novos serviços ficam disponíveis primeiro em us-east-1

Serviços regionais vs. globais

Tipo	Descrição	Exemplos
Regional	A consola AWS fica limitada a uma região selecionada — é onde o serviço é lançado	Maioria dos serviços (ex.: EC2)
Global	Opera através de múltiplas regiões, sem região fixa selecionável	S3, CloudFront, IAM

Availability Zones (AZ)

- Localização física composta por um ou mais data centers
- Uma região tem geralmente 3 AZs

- Nomenclatura: código da região + letra (ex.: us-east-1a)
- Todo o tráfego entre AZs é encriptado
- As AZs estão a até 100 km umas das outras

■■ Fault Tolerance

Conceito	Definição
Fault domain	Secção da rede vulnerável a danos se um sistema crítico falhar. O objetivo é impedir que a falha se propague para fora do domínio
Fault level	Coleção de fault domains — corresponde a uma região
Relação	Fault level = Região · Fault domain = AZ

- Cada região AWS é isolada das restantes regiões
- Cada AZ é isolada das restantes AZs, mas interligadas — cada AZ funciona como uma zona de falha independente

Porquê usar múltiplas AZs? Distribuir uma aplicação por várias AZs protege melhor contra falhas de energia, trovoadas, tornados, etc.

■ AWS Global Network

Interligações entre toda a infraestrutura global da AWS.

Point of Presence (PoP)

“Localização intermédia entre uma região AWS e o utilizador final — um data center (da AWS ou de um parceiro de confiança) usado para entrega de conteúdo ou upload acelerado.”

Tipo de PoP	Função
Edge location	Guarda cópias em cache dos ficheiros mais populares (páginas, imagens, vídeos), reduzindo a distância até ao utilizador
Regional edge cache	Guarda caches maiores de ficheiros menos populares, reduzindo round-trips e custos de transferência

Serviços que usam PoP: Amazon CloudFront, Amazon S3 Transfer Acceleration, AWS Global Accelerator

AWS Direct Connect

“Ligação privada e dedicada entre o teu data center, escritório ou colocation e a AWS.”

Direct Connect Locations — data centers de parceiros de confiança onde é possível estabelecer uma ligação dedicada, de alta velocidade e baixa latência entre o ambiente on-premise e a AWS.

AWS Local Zones

“Data centers localizados muito perto de uma área densamente povoada, para fornecer desempenho de baixíssima latência (single-digit millisecond) nessa área específica.”

AWS Wavelength Zones

“Permitem edge computing em redes 5G. As aplicações que correm em Wavelength Zones têm latência ultra-baixa, ao aproximar o processamento do utilizador final que está ligado à rede móvel.”

■■ Data Residency e Soberania de Dados

Data residency: localização física onde os dados ou recursos cloud de uma organização residem.

Conceito	Definição
Compliance boundaries	Limites de conformidade regulatória, impostos por um governo ou organização, que definem onde os dados e recursos podem residir
Data sovereignty	Controlo jurisdicional/legal que pode ser exercido sobre os dados, porque a sua localização física está dentro de determinadas fronteiras jurisdicionais

AWS Outposts: rack físico de servidores da AWS, instalado no próprio datacenter/instalações do cliente — permite correr serviços AWS on-premises, útil para cumprir requisitos de data residency.

■■ Governança e Controlo de Configuração

AWS Config: serviço de policy as code — permite criar regras que verificam continuamente a configuração dos recursos AWS. Pode gerar alertas ou despoletar ações automáticas quando deteta algo fora de conformidade.

IAM Policies: podem negar explicitamente o acesso a regiões AWS específicas.

Service Control Policies (SCPs): permissões aplicadas ao nível de toda a organização (AWS Organizations) — funcionam como um "teto" de permissões que se aplica a todas as contas abaixo, independentemente das políticas IAM locais de cada conta.

■ Notas pendentes

- [ ] Confirmar se a lista de "7 vantagens" corresponde às 6 vantagens oficiais da AWS
- [ ] Rever prints mencionados nas aulas (Tier 1, Direct Connect, Amazon Braket)
- [ ] Aprofundar diferença prática entre IAM policy (deny região) vs. SCP (nível organização)

■ Progresso: 2h00min de vídeo assistido.

Aula 3

Aula 3 — AWS para Setor Público, Sustentabilidade e Arquitetura de Alta Disponibilidade

Aula 3 — AWS para Setor Público, Sustentabilidade e Arquitetura de Alta Disponibilidade

■■ AWS for Government

Setor público inclui bens públicos e serviços governamentais como: militar, saúde, educação pública, transportes públicos.

GovCloud

Conceito	Descrição
FedRAMP	Programa do governo dos EUA que define uma abordagem padronizada para avaliação de segurança, autorização e monitorização contínua de serviços cloud
GovCloud	Regiões isoladas que um cloud provider disponibiliza especificamente para correr workloads sujeitos a FedRAMP
AWS GovCloud	Permite alojar informação sensível (Controlled Unclassified Information) — acessível apenas a entidades dos EUA e à respetiva conta root

AWS China

- Completamente isolada da AWS global — domínio próprio: amazonaws.cn
- Requer uma licença comercial chinesa para operar
- Duas regiões: Ningxia e Beijing

■ Sustentabilidade da Infraestrutura Global

- Meta: 100% energia renovável até 2025, através de:RECs — Renewable Energy CreditsGOs — Guarantees of Origin
- Eficiência energética: data centers AWS são, em média, 3,6x mais eficientes que a média dos data centers empresariais nos EUA
- Gestão de água:Tecnologia de arrefecimento por evaporação diretaUso de água não potável para arrefecimentoTratamento de água no local, para remover minerais e reutilizar a água em mais ciclosMétricas de eficiência hídrica para monitorizar o uso ótimo de água por região

■■ AWS Ground Station

Serviço totalmente gerido que permite controlar comunicações via satélite, processar dados e escalar operações sem necessidade de construir a tua própria estação terrestre.

Casos de uso: previsão meteorológica, imagem de superfície, comunicações, etc.

■ AWS Outposts

Serviço totalmente gerido que traz a mesma infraestrutura, serviços, APIs e ferramentas da AWS para (virtualmente) qualquer datacenter — na prática, é uma rack de servidores a correr infraestrutura AWS no teu próprio local físico.

Tamanho	Descrição
42U	Rack completa de servidores fornecida pela AWS
1U / 2U	Servidores individuais que podem ser inseridos nas tuas racks já existentes

“Rack: estrutura desenhada para alojar e organizar equipamento informático.”

■ Terminologia de Arquitetura Cloud

Papel	Foco
Solutions Architect	Desenha uma solução técnica usando múltiplos sistemas, através de investigação, documentação e experimentação
Cloud Architect	Foca-se exclusivamente em arquitetar soluções técnicas usando serviços cloud

Os 5 pilares que o Cloud Architect tem de garantir

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A[Availability<br/>manter o serviço disponível]
    B[Scalability<br/>crescer sem impedimentos]
    C[Elasticity<br/>encolher e crescer conforme a procura]
    D[Fault Tolerance<br/>prevenir falhas]
    E[Disaster Recovery<br/>recuperar de falhas]
```

Um Solutions Architect deve sempre considerar dois fatores de negócio:

1. Quão segura é a solução?
2. Quanto é que isto vai custar?

■ High Availability (Alta Disponibilidade)

Garantir que o serviço permanece disponível, eliminando pontos únicos de falha e/ou mantendo um certo nível de desempenho.

- Correr workloads em múltiplas AZs — se uma AZ ficar indisponível, a app continua a funcionar
- Elastic Load Balancer (ELB): distribui tráfego de forma equilibrada por múltiplos servidores, num ou mais datacenters. Se um datacenter ficar indisponível, o load balancer redireciona o tráfego para outro

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    U[Utilizadores] --> ELB[Elastic Load Balancer]
    ELB --> AZ1["AZ 1<br/>Servidor(es)"]
    ELB --> AZ2["AZ 2<br/>Servidor(es)"]
    AZ1 -.se falhar.-> ELB
    ELB -->|redireciona tráfego| AZ2
```

■ High Scalability

Capacidade de aumentar a capacidade em resposta ao aumento de tráfego, memória e poder computacional.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph Vertical["Scaling Vertical"]
        V1["Servidor pequeno"] --> V2["Servidor maior<br/>(upgrade)"]
```

```
end
subgraph Horizontal["Scaling Horizontal"]
    H1["1 servidor"] --> H2["Vários servidores<br/>do mesmo tamanho"]
end
```

Tipo	Descrição
Vertical	Upgrade para um servidor maior (mais CPU/RAM na mesma máquina)
Horizontal	Adicionar mais servidores do mesmo tamanho

■ High Elasticity

Aumentar ou reduzir a capacidade automaticamente, com base na procura atual.

- Scaling out (horizontal): adicionar mais servidores
- Scaling in (horizontal): remover servidores subutilizados
- ■■ Scaling vertical não se usa com elasticidade — só faz sentido com scaling horizontal

Auto Scaling Groups (ASG): funcionalidade da AWS que adiciona ou remove servidores automaticamente, com base em regras de scaling definidas por ti, usando métricas.

■■ Fault Tolerance

Ausência de ponto único de falha — previne a possibilidade de falha.

- Failover: plano para redirecionar tráfego para um sistema redundante, caso o sistema primário falheTer uma cópia (secundária) da base de dados, que só entra em uso se ocorrer um failover
- RDS Multi-AZ: corres uma base de dados standby duplicada noutra AZ, para o caso da base de dados primária falhar

■ High Durability

Capacidade de recuperar de um desastre e prevenir a perda de dados. A solução para isto chama-se Disaster Recovery (DR).

Perguntas-chave sobre backups:

- Fazes backup?
- Quão rápido consegues restaurar esse backup?
- O backup continua a funcionar (foi testado)?

AWS Elastic Disaster Recovery (antigo CloudEndure DR): replica continuamente as tuas máquinas para uma área de staging de baixo custo, na conta e região AWS de destino que preferires, permitindo uma recuperação rápida e fiável.

■ Business Continuity Plan (BCP)

Documento que define como o negócio vai continuar a operar durante uma interrupção não planeada dos serviços.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    I["■ Incidente"] -->|RPO: perda de dados máxima aceitável| B["Último backup válido"]
    I -->|RTO: downtime máximo tolerável| R["Serviço restaurado"]
```

Métrica	Definição
RPO (Recovery Point Objective)	Quantidade máxima de perda de dados aceitável após um incidente, expressa em tempo
RTO (Recovery Time Objective)	Tempo máximo de downtime que o negócio consegue tolerar sem sofrer uma perda financeira significativa

“■ Opções de disaster recovery — ver print da aula (backup & restore, pilot light, warm standby, multi-site ativo/ativo).”

■ Ferramentas e Conceitos de Gestão AWS

AWS API

Software através do qual duas aplicações comunicam entre si via HTTP/S.

AWS Management Console

Consola web unificada para construir, gerir e monitorizar tudo na tua conta AWS.

Service Console

Cada serviço AWS tem a sua própria consola personalizada — acessível pesquisando pelo nome do serviço.

AWS Account ID

Cada conta AWS tem um ID único, usado para:

- Login com utilizador non-root
- Acesso cross-account
- Definição de roles (ex.: dar acesso a um serviço apenas a uma conta com ID específico)
- Support cases

AWS Tools for PowerShell

- PowerShell: shell de linha de comandos e linguagem de scripting para automação e configuração de tarefas
- AWS Tools for PowerShell: permite interagir com a API da AWS através de cmdlets do PowerShell

Amazon Resource Name (ARN)

Identifica de forma única qualquer recurso AWS.

```
arn:partition:service:region:account-id:resource-id
arn:partition:service:region:account-id:resource-type/resource-id
```

exemplo: arn:aws:ec2:eu-north-1:324567123432:instance/i-0c214be854f0ca5a5

■ Progresso: 3h05min de vídeo assistido.

Aula 4

Aula 4 — Ferramentas AWS, IaC e Shared Responsibility Model

Aula 4 — Ferramentas AWS, IaC e Shared Responsibility Model

■ AWS CLI

Termo	Definição
Terminal	Interface só de texto
Console	Computador físico usado para introduzir informação fisicamente num terminal
Shell	Programa de linha de comandos com o qual o utilizador interage para inserir comandos (ex.: PowerShell, Bash)
AWS CLI	Interface de linha de comandos para interagir com a API da AWS, através de comandos de uma ou várias linhas

“■ É possível usar a AWS CLI diretamente no VS Code — basta instalar a extensão da AWS.”

AWS SDK

Software Development Kit (SDK): coleção de ferramentas de desenvolvimento de software reunidas num único pacote instalável.

O AWS SDK permite criar, modificar, apagar ou interagir com recursos AWS de forma programática (via código).

AWS CloudShell

Shell baseado no browser, embutido diretamente na AWS Management Console — não precisas de instalar nada localmente.

■■ Infrastructure as Code (IaC)

Escrever um script de configuração para automatizar a criação, atualização ou destruição de infraestrutura cloud.
Vantagens: facilita partilhar, versionar e inventariar a infraestrutura.

Ferramenta	Características
AWS CloudFormation (CFN)	Ficheiros YAML (linguagem simples), mas pode resultar em ficheiros grandes. É possível carregar um template .yml na consola do CloudFormation para validar erros antes de correr
AWS CDK (Cloud Development Kit)	Permite escrever IaC em qualquer linguagem de programação (Python, Node.js, Java, etc.). Tem biblioteca extensa, CLI própria, pipelines para configurar CI/CD rapidamente, e framework de testes para testes unitários e de integração

■ AWS Toolkit for VS Code

Plugin open-source para VS Code que permite criar, debugar e fazer deploy de recursos AWS diretamente no editor.

Funcionalidade	Descrição
AWS Explorer	Explora uma grande variedade de recursos AWS associados à tua conta

Funcionalidade	Descrição
AWS CDK Explorer	Explora as tuas stacks definidas com o CDK
Amazon ECS support	Fornece IntelliSense (autocompletar + descrições) para ficheiros de task definition do ECS
Serverless Applications	Cria, debuga e faz deploy de aplicações serverless

■ Access Keys

Chave (access key) + segredo (secret key), necessários para ter acesso programático aos recursos AWS ao interagir com a API fora da Management Console.

- Também chamadas de AWS credentials
- Cada utilizador pode ter 2 chaves ativas em simultâneo
- Podem ser desativadas quando necessário
- ■■ Quem usa a chave tem exatamente as mesmas permissões do utilizador associado a ela
- Guardadas localmente em ~/.aws/credentials

AWS Documentation

Grande coleção de documentação técnica sobre como usar os serviços AWS — a fonte de referência oficial para tudo.

■■ Shared Responsibility Model

Framework de segurança cloud que define as obrigações de segurança do cliente vs. do cloud provider (ex.: AWS).

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph AWS["■■ AWS — Security OF the Cloud"]
        A1["Hardware e infraestrutura global<br/>(Regions, AZs, Edge Locations, segurança física)"]
        A2["Software subjacente<br/>(compute, storage, database, rede física)"]
    end
    subgraph CUSTOMER["■■■ Cliente — Security IN the Cloud"]
        C1["Configuração de serviços geridos<br/>e software de terceiros<br/>(plataformas, apps, IAM)"]
        C2["Configuração de infraestrutura<br/>e sistemas virtuais<br/>(SO, rede, firewall)"]
        C3["Configuração de segurança dos dados<br/>(encriptação client/server-side,<br/>proteção do tráfego de rede, dados do c
    end
```

“■ AWS é responsável pela segurança DA cloud. ■ O cliente é responsável pela segurança NA cloud.”

Regra prática (das duas definições do print):

- Responsabilidade "IN the cloud": se conseguires configurar ou armazenar algo, tu (cliente) és responsável por isso.
- Responsabilidade "OF the cloud": se não conseguires configurar algo, a AWS é responsável por isso.

■ Tipos de Responsabilidade na Cloud Computing

Como a responsabilidade se distribui ao longo da stack, consoante o modelo de serviço:

Camada	On-Premise	IaaS	PaaS	SaaS
Applications	■ Cliente	■ Cliente	■ Cliente	■ AWS
Data	■ Cliente	■ Cliente	■ Cliente	■ AWS
Runtime	■ Cliente	■ Cliente	■ AWS	■ AWS

Camada	On-Premise	IaaS	PaaS	SaaS
Middleware	■ Cliente	■ Cliente	■ AWS	■ AWS
OS	■ Cliente	■ Cliente	■ AWS	■ AWS
Virtualization	■ Cliente	■ AWS	■ AWS	■ AWS
Servers	■ Cliente	■ AWS	■ AWS	■ AWS
Storage	■ Cliente	■ AWS	■ AWS	■ AWS
Networking	■ Cliente	■ AWS	■ AWS	■ AWS

“■ Padrão a memorizar: On-Premise → tudo é teu. IaaS → tu geres a partir do SO para cima. PaaS → tu geres só a app e os dados. SaaS → só usas, não configuras nada.”

■■ Shared Responsibility Model — Compute

À medida que avanças de Bare Metal para Functions, a AWS assume progressivamente mais responsabilidade e tu (cliente) cada vez menos:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    BM["Bare Metal<br/>EC2 Bare Metal Instance"] --> D["Dedicated<br/>EC2 Dedicated Instance/Host"]
    D --> VM["Virtual Machines<br/>EC2"]
    VM --> C["Containers<br/>ECS / EKS / Fargate"]
    C --> F["Functions<br/>AWS Lambda"]
```

Camadas afetadas ao longo do espectro: Código → App Container → Runtime → OS → Virtualização.

- Código: é sempre responsabilidade do cliente, em qualquer ponto do espectro.
- Virtualização: é (quase) sempre responsabilidade da AWS, em qualquer ponto do espectro.
- App Container, Runtime, OS: a responsabilidade vai-se transferindo do cliente para a AWS à medida que avanças de Bare Metal para Functions.

Comparação detalhada por tipo de compute

Tipo	Exemplo AWS	Responsabilidade do Cliente	Responsabilidade da AWS
IaaS — Bare Metal	EC2 Bare Metal Instance	Configuração do Host OS, Hypervisor	Máquina física
IaaS — Virtual Machine	EC2	Configuração do Guest OS, runtime de containers	Hypervisor, máquina física
IaaS — Containers	Amazon ECS	Configuração, deployment e storage dos containers	OS, hypervisor, runtime de containers
PaaS	AWS Elastic Beanstalk	Upload do código, configuração do ambiente, estratégias de deployment, configuração de serviços associados	Servidores, OS, rede, storage, segurança
SaaS	Amazon WorkDocs	Conteúdo dos documentos, gestão de ficheiros, configuração de partilha/acesso	Servidores, OS, rede, storage, segurança
FaaS	AWS Lambda	Apenas o upload do código	Praticamente tudo: deployment, runtime, rede, storage, segurança, máquina física

■■ Shared Responsibility Model — Arquitetura

Quanto mais próximo do serverless, menos responsabilidade tens — mas também menos controlo direto:

Nível de responsabilidade	Estilo de arquitetura	Serviços AWS	Características
■ Menos responsabilidade	Serverless / Functions	Lambda, Amplify	Não há servidores a gerir — só te preocupas com dados e código
■ Responsabilidade intermédia	Microservices / Containers	Fargate, ECS, EKS	Combinas linguagens diferentes, melhor aproveitamento de recursos
■ Mais responsabilidade	Traditional / VMs	Elastic Beanstalk (PaaS), EC2 (IaaS)	Arquitetura mais familiar globalmente — muita documentação, frameworks e suporte disponíveis

■ Shared Responsibility Model — Detalhe por Camada e Modelo

Visualização que ajuda a determinar exatamente o que o cliente e o que a AWS (CSP) são responsáveis, por camada e por modelo de serviço:

Camada	IaaS	PaaS	SaaS
Content	■ Cliente	■ Cliente	■ Cliente
Access Policies	■ Cliente	■ Cliente	■ Cliente
Usage	■ Cliente	■ Cliente	■ Cliente
Deployment	■ Cliente	■ Cliente	■ AWS
Web application security	■ Cliente	■ Cliente	■ AWS
Identity	■ Cliente	■ AWS	■ AWS
Operations	■ Cliente	■ AWS	■ AWS
Access and authentication	■ Cliente	■ AWS	■ AWS
Network security	■ Cliente	■ AWS	■ AWS
Guest OS, data & content	■ Cliente	■ AWS	■ AWS
Audit logging	■ AWS	■ AWS	■ AWS
Network	■ AWS	■ AWS	■ AWS
Storage + encryption	■ AWS	■ AWS	■ AWS
Hardened Kernel + IPC	■ AWS	■ AWS	■ AWS
Boot	■ AWS	■ AWS	■ AWS
Hardware	■ AWS	■ AWS	■ AWS

“■ Repara no padrão: à medida que avanças de IaaS → PaaS → SaaS, a "linha divisória" sobe — a AWS absorve cada vez mais camadas, e o cliente fica só com o topo (conteúdo, políticas de acesso e utilização).”

■ Progresso: 4h20min de vídeo assistido.

Aula 5

Aula 5 — Computing Services

Aula 5 — Computing Services

■■ Mapa geral dos serviços de compute

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    C[AWS Compute] --> VM["Máquinas virtuais<br/>EC2 · Lightsail"]
    C --> CT["Containers<br/>ECS · Fargate · EKS · ECR"]
    C --> SL["Serverless<br/>Lambda"]
    C --> HPC["Alta performance<br/>Nitro · Bare Metal · Bottlerocket · ParallelCluster"]
    C --> EH["Edge e Hybrid<br/>Outposts · Wavelength · VMware Cloud · Local Zones"]
    C --> CM["Custo e capacidade<br/>Spot/Reserved/Savings Plans · Batch · Compute Optimizer · ASG · ELB · Beanstalk"]
```

■ Amazon EC2 (Elastic Compute Cloud)

Lança uma máquina virtual, chamada de instância.

- Máquina virtual: emulação de um computador físico através de software.
- Virtualização de servidores: permite criar, copiar, redimensionar ou migrar servidores facilmente.

EC2 é um servidor altamente configurável. Podes escolher:

Opção	Exemplo
AMI (Amazon Machine Image)	Imagem base com SO e software
CPU e RAM	Tamanho da instância
Largura de banda de rede	Desempenho de rede
Sistema operativo	Linux, Windows, etc.
Armazenamento	Vários discos virtuais anexados (ex.: EBS — Elastic Block Store)

■ Serviços de Compute

Serviço	O que é
Amazon Lightsail	Servidor virtual gerido; a versão "amigável" do EC2
Containers	Virtualização do sistema operativo para correr múltiplos workloads numa única instância
ECS (Elastic Container Service)	Serviço de orquestração de containers com suporte a Docker. Lança um cluster de servidores em instâncias EC2 com Docker instalado
ECR (Elastic Container Registry)	Repositório de imagens de containers, com controlo de versões
ECS Fargate	Orquestração de containers serverless. Igual ao ECS, mas pagas por container em execução
EKS (Elastic Kubernetes Service)	Kubernetes totalmente gerido. Kubernetes é software open-source de orquestração criado pela Google, geralmente o padrão para gerir microservices
Lambda	Serviço de funções serverless: corres código sem provisionar ou gerir servidores

ECS vs. Fargate

	ECS (com EC2)	ECS com Fargate
Servidores	Tens de manter um servidor EC2 ligado, mesmo sem containers a correr	A AWS gere o servidor subjacente
Escalar/atualizar	Por tua conta	Não precisas de escalar nem atualizar o EC2
Pagamento	Pelo servidor EC2	On-demand, por container em execução

Lambda

Fazes upload de um pequeno pedaço de código e defines:

- Quanta memória usa
- Quanto tempo a função pode correr antes de dar timeout

■■ Na prática: ECS e Lambda

ECS

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["Imagem<br/>(ECR ou Docker Hub)"] --> B[Create cluster]
    B --> C{Como correr?}
    C -->|Create Service| D["Funciona como uma app<br/>(sempre a correr)"]
    C -->|Run Task| E["Corre e pára<br/>(ex: executar um comando)"]
```

Task definition — é preciso definir:

- Um nome
- CPU e memória para a task
- A imagem do container (Docker Hub ou ECR)
- Portas a abrir

Lambda

1. Escolhes a linguagem do código
2. Cola o código (simples) ou carrega um .zip do projeto ou a partir de um bucket S3
3. Clica Deploy
4. Testa

■ Serviços de Alta Performance

Nitro System

Hardware dedicado + hypervisor leve. Permite inovação mais rápida e segurança reforçada.

Componente	Função
Nitro Cards	Placas especializadas para VPC, EBS e instance storage
Nitro Security Chips	Integrados na motherboard, protegem os recursos de hardware
Nitro Hypervisor	Hypervisor leve, gere a alocação de memória e CPU

Outros serviços

Serviço	Descrição
Bare Metal Instance	Instância EC2 sem hypervisor: corres workloads diretamente no hardware, para máxima performance e controlo (ex.: famílias M5 e R5 têm variantes bare metal)
Bottlerocket	SO Linux open-source, criado pela AWS especificamente para correr containers em VMs ou bare metal
AWS ParallelCluster	Ferramenta open-source de gestão de clusters, facilita o deploy e gestão de clusters de high performance computing (HPC) na AWS

■ Edge e Hybrid Computing

- Edge computing: empurras os workloads para fora da tua rede, para correrem perto do destino/utilizador final.
- Hybrid computing: corres workloads tanto no teu datacenter on-premise como numa AWS VPC.

Serviço	Descrição
AWS Outposts	Rack física de servidores que colocas no teu datacenter
AWS Wavelength	Construir e lançar aplicações num datacenter de telecom, com latência ultra-baixa via rede 5G, o mais perto possível do utilizador
VMware Cloud on AWS	Gerir máquinas virtuais VMware on-premise a correr na AWS. O datacenter tem de usar VMware para virtualização
AWS Local Zones	Datacenters edge fora das regiões AWS, para usar a AWS mais perto do destino final

■ Gestão de Custo e Capacidade

- Cost management: como poupar dinheiro?
- Capacity management: como responder à procura de tráfego e utilização, adicionando ou melhorando servidores?

Formas de poupar em compute

EC2 Spot Instances, Reserved Instances e Savings Plans: formas de poupar em compute pagando adiantado (total ou parcialmente), comprometendo-te com contratos anuais, ou sendo flexível quanto à disponibilidade e interrupções do serviço.

Outros serviços

Serviço	Descrição
AWS Batch	Planeia, agenda e executa workloads de batch computing em toda a gama de serviços de compute da AWS
AWS Compute Optimizer	Sugere como reduzir custos e melhorar performance, usando machine learning para analisar o teu histórico de utilização
EC2 Auto Scaling Groups (ASG)	Adiciona ou remove servidores EC2 automaticamente conforme a procura. Poupas dinheiro e cumpres a capacidade, porque só corres os servidores de que precisas
Elastic Load Balancer (ELB)	Distribui tráfego por várias instâncias, redireciona de instâncias com falhas (unhealthy) para saudáveis, e pode distribuir por diferentes AZs
AWS Elastic Beanstalk (EB)	Facilita o deploy de aplicações web sem que os developers tenham de configurar ou compreender os serviços AWS subjacentes

■ Progresso: 5h12min de vídeo assistido.

Aula 6

Aula 6 — Storage e Databases

Aula 6 — Storage e Databases

■ PARTE 1 — STORAGE

Tipos de storage: qual escolher?

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    Q{O que precisas?} -->|Guardar ficheiros,<br/>sem gerir infraestrutura| S3["Amazon S3<br/>Object Storage"]
    Q -->|Disco virtual<br/>para UMA VM| EBS["Amazon EBS<br/>Block Storage"]
    Q -->|Partilha de ficheiros entre<br/>VÁRIAS VMs/utilizadores| EFS["Amazon EFS<br/>File Storage"]
```

Serviço	Quando usar	Características
S3	Fazer upload de ficheiros, sem te preocupares com a infraestrutura subjacente	Não indicado para IOPS elevados; sem limite de número de ficheiros; suporta múltiplas leituras e escritas em simultâneo
EBS	Precisas de um disco rígido virtual anexado a uma VM	Suporta apenas um volume de escrita (uma instância de cada vez); acesso direto pelo SO
EFS	Vários utilizadores ou VMs precisam de aceder à mesma drive (shared folders)	Ligações múltiplas via rede; suporta múltiplas leituras; a escrita bloqueia o ficheiro (file locking)

Amazon S3

Object storage (armazenamento por objetos): gere os dados como objetos, ao contrário de outras arquiteturas de storage:

Arquitetura	Como organiza os dados
File systems	Ficheiros organizados numa hierarquia de pastas
Block storage	Dados em blocos, dentro de setores e tracks (como um disco rígido)
Object storage (S3)	Dados como objetos independentes, sem hierarquia real

“S3 tem storage ilimitado. A consola do S3 fornece uma interface para fazeres upload e aceder aos dados.”

Anatomia de um objeto S3

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    O["■ Objeto S3"] --> K["Key<br/>(nome do objeto)"]
    O --> V["Value<br/>(os dados em si)"]
    O --> VER["Version ID"]
    O --> M["Metadata<br/>(informação adicional)"]
```

Bucket

- Bucket: contentor que guarda objetos (e pode ter "pastas" que, por sua vez, guardam objetos).
- Tem um namespace universal — o nome do bucket tem de ser único a nível global, em toda a AWS.
- Storage ilimitado no bucket, mas cada objeto individual tem um limite de 5 TB.

S3 Storage Classes

A AWS oferece várias classes de storage, que trocam velocidade de acesso e disponibilidade por custo mais baixo:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    A["■ S3 Standard<br/>(mais caro, acesso mais rápido)"]
    B["S3 Intelligent-Tiering"]
    C["S3 Standard-IA<br/>(Infrequent Access)"]
    D["S3 One Zone-IA"]
    E["S3 Glacier"]
    F["■ S3 Glacier Deep Archive<br/>(mais barato, acesso mais lento)"]
    A --> B --> C --> D --> E --> F
```

Classe	Ideal para
S3 Standard	Dados acedidos frequentemente
S3 Intelligent-Tiering	Padrões de acesso desconhecidos ou imprevisíveis — a AWS move os dados automaticamente entre camadas
S3 Standard-IA	Dados acedidos com pouca frequência, mas que precisam de acesso rápido quando necessário
S3 One Zone-IA	Como o Standard-IA, mas guardado numa única AZ (mais barato, menos resiliente)
S3 Glacier	Arquivo a longo prazo, recuperação em minutos/horas
S3 Glacier Deep Archive	Arquivo muito raramente acedido, recuperação em horas — a opção mais barata de todas

AWS Snow Family

Dispositivos de storage e compute usados para mover fisicamente dados de/para a cloud, quando a transferência pela rede seria demasiado lenta.

Dispositivo	Tamanho	Capacidade
Snowcone	Muito pequeno (tipo mala pequena)	8 TB HDD / 14 TB SSD
Snowball Edge	Tamanho de mala	Storage optimized: 80 TB · Compute optimized: 39,5 TB
Snowmobile	Contentor transportado por camião semirreboque	100 PB por reboque

“■ Padrão a memorizar: Snowcone (mini) → Snowball/Snowball Edge (mala, dezenas de TB) → Snowmobile (contentor inteiro, petabytes).”

Serviços de Storage — resumo detalhado

Amazon S3

Serviço de object storage serverless. Uploads de ficheiros grandes, quantidade ilimitada de ficheiros. Pagas pelo que armazenas, sem te preocupares em aumentar o tamanho do disco ou gerir o sistema de ficheiros subjacente.

- URLs pré-assinados: podes gerar um URL para partilhar dados com outras pessoas, definindo um tempo de expiração para o download.
- Bucket policy: define quem pode (ou não) aceder, e a que ficheiros.
- Lifecycle rules: ações automáticas sobre os objetos, por exemplo:Apagar versões não-atuais de objetosMover versões atuais para outras storage classesApagar objetos expirados

Amazon S3 Glacier

Serviço de cold storage — armazenamento de baixo custo para arquivo e backups de longo prazo. Usa discos HDD; altamente seguro e durável.

Amazon EBS (Elastic Block Store)

Serviço de block storage persistente — disco rígido virtual na cloud, anexado a instâncias EC2. Vários tipos de disco disponíveis: SSD, IOPS SSD, HDD, etc.

Amazon EFS (Elastic File System)

Serviço de sistema de ficheiros NFS nativo da cloud. Storage de ficheiros que podes montar em múltiplas instâncias EC2 — usa quando precisas de partilhar ficheiros entre vários servidores.

“Fluxo típico: criar um access point e configurá-lo na EC2 → fica disponível na pasta /mnt.”

AWS Storage Gateway

Serviço de storage híbrido que estende o teu storage on-premise para a cloud.

Tipo de gateway	Função
File Gateway	Estende o teu storage local para o Amazon S3
Volume Gateway	Faz cache das tuas drives locais no S3, mantendo um backup contínuo dos ficheiros locais na cloud
Tape Gateway	Guarda ficheiros em tapes virtuais, para backups a longo prazo com custo muito reduzido

AWS Backup

Serviço de backup gerido, que centraliza e automatiza o backup de dados em vários serviços AWS: EC2, EBS, RDS, DynamoDB, EFS, Storage Gateway.

AWS Elastic Disaster Recovery (antigo CloudEndure DR)

Replica continuamente as tuas máquinas para uma área de staging de baixo custo, na conta e região AWS de destino, permitindo uma recuperação rápida e fiável.

Amazon FSx

Sistemas de ficheiros ricos em funcionalidades e de alta performance.

Variante	Descrição
FSx for Windows File Server	Usa o protocolo SMB, permite montar em servidores Windows
FSx for Lustre	Usa o sistema de ficheiros Lustre (Linux), permite montar em servidores Linux — muito usado em workloads de HPC/machine learning

■■ PARTE 2 — DATABASES

Visão geral

Uma base de dados guarda dados estruturados ou semiestruturados, usando técnicas formais de design e modelação.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    D[Databases] --> R["Relacionais<br/>dados tabulares fortemente estruturados"]
    D --> NR["Não-relacionais (NoSQL)<br/>dados semiestruturados"]
```

```
R --> RO["Row-oriented<br/>(orientada a linhas)"]
R --> CO["Column-oriented<br/>(orientada a colunas)"]
```

Tipo	Descrição
Relacional	Dados estruturados, fortemente tabulares. Pode ser orientada a linhas ou a colunas
Não-relacional (NoSQL)	Dados semiestruturados, que podem ou não parecer-se remotamente com uma tabela

Data Warehouse

Desenhado para workloads analíticos — normalmente um data store orientado a colunas.

- Faz agregação dos dados (agrupar dados).
- Colunas otimizadas para leitura.
- É "HOT": devolve resultados de queries muito rapidamente.

Key-Value Store (explicado em detalhe)

Um key-value store é um tipo de base de dados não-relacional (NoSQL) que guarda dados de forma extremamente simples: a cada chave (key) única corresponde um valor (value) — normalmente um blob opaco de dados (texto, JSON, binário).

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph KV["Key-Value Store"]
        K1["user:123"] --> V1["{ nome: 'Luis', idade: 23 }"]
        K2["session:abc"] --> V2["{ token: 'xyz789' }"]
        K3["cart:456"] --> V3["[ 'item1', 'item2' ]"]
    end
```

Como funciona na prática: para ler um valor, dás a chave exata (ex.: GET user:123) e recebes o valor associado — não há "procurar por partes do valor" nem relações entre chaves.

Característica	Explicação
■ Rápido	Como a pesquisa é feita diretamente pela chave (como um dicionário/hash map), as operações são extremamente rápidas — normalmente O(1)
■ "Dumb" (simples)	Não entende a estrutura do valor — para o key-value store, o valor é só um bloco de dados opaco
■ Escala facilmente	Fácil de distribuir por muitos servidores, porque não há relações complexas a manter entre registos
■ Sem relações	Não existem "joins" entre diferentes conjuntos de dados, como numa base de dados relacional
■ Sem índices secundários avançados	Normalmente só conseguem pesquisar pela chave principal, não por campos dentro do valor
■ Sem agregações nativas	Operações como SUM, GROUP BY, COUNT não são o forte deste tipo de base de dados

Quando usar: sessões de utilizador, carrinhos de compras, caches, perfis simples de utilizador — sempre que o acesso é feito quase sempre "dá-me o valor desta chave" e não precisas de fazer perguntas complexas aos dados.

Exemplo AWS: Amazon DynamoDB (no modo key-value) e Amazon ElastiCache (Redis/Memcached).

Document Store

Base de dados NoSQL que guarda documentos como estrutura de dados principal. Um documento pode ser XML, JSON, ou "JSON-like" (formato semelhante ao JSON).

“Ao contrário do key-value store puro, um document store entende a estrutura interna do documento — por isso consegue fazer queries a campos específicos dentro do documento (ex.: "todos os utilizadores com idade > 18").”

Serviços de Base de Dados NoSQL

Serviço	Descrição	Quando usar
DynamoDB	Base de dados NoSQL serverless, key-value e document. Desenhada para escalar a milhares de milhões de registos, com retorno de dados consistente em menos de um segundo. Não te preocupas em gerir shards	Quando queres uma base de dados massivamente escalável, económica e muito rápida
DocumentDB	Base de dados de documentos NoSQL, compatível com MongoDB	Quando queres uma base de dados tipo MongoDB, gerida pela AWS
Amazon Keyspaces	Apache Cassandra totalmente gerido	Quando queres usar Apache Cassandra sem gerir a infraestrutura

“■ DynamoDB — PartiQL editor: permite escrever comandos tipo SQL para filtrar e obter dados, por exemplo:”

Serviços de Base de Dados Relacional

Amazon RDS (Relational Database Service)

O tipo de base de dados mais usado. Suporta múltiplos motores SQL: MySQL, MariaDB, PostgreSQL, Oracle, Microsoft SQL Server, Aurora.

Aspeto configurável	Detalhe
Gestão de credenciais	Auto-gerida — password master
Configuração da instância	Quantos CPU, RAM, largura de banda de rede
Storage	Tipo de disco (SSD, etc.)
Ligação à EC2	Opcional — ligar ou não uma instância EC2 à base de dados
RDS Proxy	Usado, por exemplo, pelo Lambda, para gerir pedidos de pull/push à base de dados de forma eficiente

Amazon Aurora

Base de dados totalmente gerida — 5x mais rápida que o MySQL e 3x mais rápida que o PostgreSQL.

“Usa quando queres uma base de dados relacional altamente disponível, durável, escalável e segura, compatível com PostgreSQL ou MySQL.”

Aurora Serverless

Versão on-demand e serverless do Aurora.

“Usa quando queres a maioria dos benefícios do Aurora, mas estás disposto a aceitar cold starts, ou não tens tráfego elevado constante.”

RDS on VMware

Permite fazer deploy dos motores suportados pelo RDS num datacenter on-premise. O datacenter tem de usar VMware para virtualização.

“Usa quando queres bases de dados geridas pelo RDS, mas no teu próprio datacenter.”

Outros serviços de bases de dados

Serviço	Tipo	Quando usar
Amazon Redshift	Data warehouse à escala de petabytes	Gerar analytics ou relatórios rapidamente a partir de grandes volumes de dados
Amazon ElastiCache	Base de dados em memória / caching (Redis ou Memcached), gerida	Melhorar a performance de uma aplicação, adicionando uma camada de cache à frente do servidor web ou da base de dados
Amazon Neptune	Base de dados de grafos gerida	Compreender ligações entre dados — ex.: mapear redes de fraude ou relações em redes sociais
Amazon Timestream	Base de dados de séries temporais gerida	Medir como algo muda ao longo do tempo
Amazon QLDB (Quantum Ledger Database)	Base de dados de livro-razão (ledger) gerida — logs de transações transparentes, imutáveis e criptograficamente verificáveis	Registar histórico de atividades financeiras que precisa de ser confiável
AWS DMS (Database Migration Service)	Serviço de migração de bases de dados	On-premise → AWS; entre duas bases de dados (mesma ou diferente conta AWS) com motores SQL diferentes; ou até de SQL para NoSQL

■ Progresso: 6h28min de vídeo assistido.

Aula 7

Aula 7 — Networking e EC2 em Detalhe

Aula 7 — Networking e EC2 em Detalhe

■ PARTE 1 — NETWORKING

Serviços de rede nativos da cloud

Serviço	O que faz
VPC (Virtual Private Cloud)	Secção logicamente isolada da AWS onde lanças os teus recursos
Internet Gateway (IGW)	Permite o acesso à internet
Route Tables	Determinam para onde é encaminhado o tráfego de rede das subnets
Region	Localização geográfica da tua rede
AZ (Availability Zone)	O(s) datacenter(s) onde os teus recursos AWS residem
Subnet	Partição lógica de uma rede IP em segmentos mais pequenos
NACL	Firewall ao nível da subnet
Security Group	Firewall ao nível da instância

Diagrama de rede

ver print diagrama de rede

“■ Repara na hierarquia: Conta AWS → Region → VPC → Availability Zone → Subnet (pública/privada) → NACL → Route Table → Security Group → Recurso (EC2, RDS, ...). O tráfego que vem da internet passa sempre pelo Internet Gateway antes de chegar a uma subnet pública.”

Enterprise / Hybrid Networking

Serviço	Função
AWS VPN (Virtual Private Network)	Ligação segura entre o on-premise, escritórios remotos e colaboradores móveis
AWS Direct Connect	Ligação dedicada de gigabits, do datacenter on-premise até à AWS — muito rápida
AWS PrivateLink (VPC Interface Endpoints)	Mantém o tráfego dentro da rede AWS, sem atravessar a internet pública — mais seguro

VPC e Subnets

VPC: secção logicamente isolada da AWS onde lanças recursos. Escolhes um intervalo de IPs através de um CIDR range.

```
Exemplo: 10.0.0.0/16 → 65.536 endereços IP
```

Subnets: dividem o intervalo de IPs da VPC em redes mais pequenas. Uma subnet tem de ter um intervalo mais pequeno do que o da VPC.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    VPC["VPC: 10.0.0.0/16<br/>65.536 IPs"] --> S1["Subnet 1: 10.0.0.0/24<br/>256 IPs"]
    VPC --> S2["Subnet 2: 10.0.1.0/24<br/>256 IPs"]
```

Tipo de subnet	Característica
Pública	Consegue chegar à internet
Privada	Não consegue chegar à internet (a não ser via NAT)

Security Groups vs. NACLs

	Security Group	NACL
Nível	Instância	Subnet
Regras	Só allow (nega tudo por defeito)	Allow e deny
Estado	Stateful (uma resposta a tráfego permitido é sempre permitida)	Stateless (é preciso permitir explicitamente o tráfego de ida e de volta)
Uso típico	Abrir uma porta específica numa instância	Bloquear um IP problemático a nível de rede

Security Group — regras de entrada (inbound), exemplos

Regra	Type	Protocol	Port Range	Source	Motivo
Acesso SSH	SSH	TCP	22	O teu IP (/32)	Administração remota, só a partir do teu IP
Acesso a base de dados	PostgreSQL	TCP	5432	Security group da app (ou IP específico)	Só a aplicação (não o mundo) acede à BD
Website	HTTPS	TCP	443	0.0.0.0/0	Qualquer visitante acede ao site

“As regras de saída (outbound) normalmente não se mexem — mantém-se a configuração por defeito, que permite todo o tráfego em todas as portas, para a instância poder atualizar pacotes, aceder a APIs, etc.”

NACL — regras, exemplos

Rule #	Type	Protocol	Port Range	Source	Allow/Deny	Motivo
100	HTTP	TCP	80	0.0.0.0/0	ALLOW	Permitir tráfego web de entrada
110	Custom	TCP	Todas	203.0.113.55/32	DENY	Bloquear um IP conhecido por atividade abusiva
* (default)	Todo o tráfego	Todos	Todas	0.0.0.0/0	DENY	Regra final: nega tudo o que não corresponde a nenhuma regra anterior

“■ Os números das regras normalmente ficam entre 100 e 200 (com espaço entre eles), para ser fácil inserir novas regras no meio mais tarde sem ter de renumerar tudo. As NACLs avaliam as regras por ordem numérica crescente e param na primeira que corresponder.”

■■ Hands-on: criar uma subnet pública

Passos seguidos na aula:

1. Criar uma VPC

2. Criar uma subnet dentro dessa VPC
3. Criar um Internet Gateway
4. Anexar o Internet Gateway à VPC (não à subnet diretamente — o IGW liga-se à VPC como um todo)
5. Editar a Route Table da subnet: adicionar uma rota nova com destino 0.0.0.0/0 e target o Internet Gateway → Isto torna a subnet pública
6. Ativar "Auto-assign public IPv4" na subnet (para que instâncias lançadas nela recebam automaticamente um IP público)
7. Ao criar uma EC2, associá-la a esta VPC/subnet

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["1. Criar VPC"] --> B["2. Criar Subnet"]
    B --> C["3. Criar Internet Gateway"]
    C --> D["4. Anexar IGW à VPC"]
    D --> E["5. Editar Route Table<br/>0.0.0.0/0 → IGW"]
    E --> F["6. Auto-assign public IPv4"]
    F --> G["■ Subnet pública pronta"]
```

■■ PARTE 2 — EC2 EM DETALHE

EC2 — recapitulação

Servidor virtual altamente configurável:

- Capacidade de computação redimensionável
- Rápido de lançar

Escolhas ao criar uma instância:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    EC2["Configurar EC2"] --> AMI["SO via AMI<br/>(Amazon Machine Image)"]
    EC2 --> TYPE["Tipo de instância<br/>(t2.nano, t3.micro, ...)"]
    EC2 --> STOR["Storage<br/>(SSD, HDD, fita virtual, múltiplos volumes)"]
    EC2 --> CONF["Configuração da instância<br/>(key pairs, security groups,<br/>user data, IAM roles, placement groups)"]
```

Famílias de instâncias EC2

Combinações diferentes de CPU, memória, storage e capacidade de rede, otimizadas para diferentes tipos de workload:

Família	Exemplos	Ideal para	Casos de uso típicos
General Purpose	A1, T2, T3, T3a, T4g, M4, M5, M5a, M5n, M6zn, M6g, M6i, Mac	Equilíbrio entre compute, memória e rede	Servidores web, repositórios de código
Compute Optimized	C5, C4, Cba, C5n, C6g, C6gn	Aplicações limitadas pelo processador, que beneficiam de CPUs de alta performance	Modelação científica, servidores de jogos dedicados, motores de anúncios
Memory Optimized	R4, R5, R5a, R5b, R5n, X1, X1e, High Memory, z1d	Performance rápida para workloads que processam grandes datasets em memória	Caches em memória, bases de dados em memória, analytics de big data em tempo real
Accelerated Computing	P2, P3, P4, G3, G4ad, G4dn, F1, Inf1, VT1	Aceleradores de hardware ou co-processadores	Machine learning, finanças computacionais, análise sísmica, reconhecimento de voz
Storage Optimized	I3, I3en, D2, D3, D3en, H1	Acesso sequencial de leitura/escrita muito rápido a grandes datasets em storage local	Bases de dados NoSQL, bases de dados em memória/transacionais, data warehousing

“■ Padrão a memorizar pela letra inicial: T/M = general purpose, C = compute, R/X = memória (RAM), P/G/F/Inf = aceleração (GPU/FPGA), I/D/H = storage.”

Tipos e tamanhos de instância (dentro de uma família)

Tipo de instância = família + tamanho específico. Os tamanhos costumam seguir esta progressão:

```
nano → micro → small → medium → large → xlarge → 2xlarge → ... → metal
```

“metal = instância bare metal (sem hypervisor, acesso direto ao hardware).”

Exemplo — família T3

Tipo	vCPUs	Memória (GiB)	preço por hora com o linux
t3.nano	2	0.5	0.0052
t3.micro ■ Free tier	2	1	0.0108
t3.small	2	2	0.0216
t3.medium	2	4	0.0432
t3.large	2	8	0.0864

\ O que é importante fixar é o padrão: ao subir de tamanho dentro da mesma família, o preço por hora tende a duplicar a cada salto, e os atributos-chave (RAM) também tendem a duplicar.*

Dedicated Host vs. Dedicated Instance

Duas formas de teres isolamento físico de outros clientes AWS no mesmo hardware:

	Dedicated Instance	Dedicated Host
Isolamento	Isolamento da instância	Isolamento do servidor físico completo
Faturação	Por instância (+ $2 taxa por região)	Por host físico
Visibilidade de características físicas	Nenhuma visibilidade	Vês sockets, cores, host ID
Afinidade host↔instância	Sem afinidade (pode mudar de servidor físico)	Consistência — consegues fazer deploy sempre no mesmo servidor físico
Controlo de posicionamento	Sem controlo	Controlo adicional sobre onde a instância fica posicionada no servidor físico
Posicionamento automático	Sim	Sim
Adicionar capacidade via allocation request	Não	Sim

“■ Usa Dedicated Host quando precisas de trazer as tuas próprias licenças de software que dependem de características físicas do servidor (ex.: licenciamento por socket/core — BYOL, bring your own license). Usa Dedicated Instance quando só precisas de isolamento de instância, sem te importares com o servidor físico exato.”

EC2 Tenancy (modelo de ocupação)

Existem 3 opções de tenancy para uma instância EC2:

Tenancy	Onde vive a instância	Partilha de hardware físico
Default (Shared)	Em hardware partilhado, gerido pela AWS	Pode partilhar o servidor físico com instâncias de outras contas AWS

Tenancy	Onde vive a instância	Partilha de hardware físico
Dedicated Instance	Em hardware reservado só para a tua conta	O servidor físico só corre instâncias da tua conta, mas a AWS escolhe qual servidor, e pode mudar entre reinícios
Dedicated Host	Num servidor físico específico, que tu controlas	Tens visibilidade e controlo total sobre esse servidor físico (sockets, cores, posicionamento das instâncias)

Correção ao apontamento original: não é bem que "a instância default vive lá até reiniciar" — o que muda com o reinício (stop/start, não reboot) é tipicamente o IP público e, nalguns casos, o servidor físico subjacente; a tenancy em si (default/dedicated) é uma propriedade que se define na criação da instância e não muda sozinha com um simples restart.

“■ Resumindo a progressão de isolamento: Default (partilhado com outras contas) → Dedicated Instance (só a tua conta, mas sem controlo do servidor exato) → Dedicated Host (controlo total de um servidor físico específico).”

EC2 e Elastic IP

Se parares uma instância EC2 e a voltares a iniciar, o endereço IP público muda.

Solução: um Elastic IP — crias um IP fixo e associa-lo à instância. O IP mantém-se estático, mesmo que pares e inicies a instância várias vezes.

“■ Um Elastic IP não associado a nenhuma instância em execução pode gerar custo — a AWS cobra por IPs elásticos "parados" para incentivar a libertação de endereços não usados.”

AMI vs. Launch Template (correção ao apontamento original)

No teu apontamento havia uma troca de conceitos — aqui fica a explicação correta:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph AMI_FLOW["Amazon Machine Image (AMI)"]
        EC2A["EC2 existente<br/>(SO + apps + configuração instalada)"] -->|"criar imagem"| AMI["AMI<br/>(fotografia/snapshot com
        AMI -->|"lançar nova instância"| EC2B["Nova EC2 idêntica"]
    end
```

Conceito	O que realmente é
AMI (Amazon Machine Image)	Uma imagem/fotografia de uma instância EC2 num dado momento — inclui o sistema operativo, as aplicações instaladas e a configuração. Se terminares a EC2 original, consegues lançar uma nova instância a partir da AMI, com exatamente o mesmo estado que a original tinha quando a imagem foi criada
Launch Template	Uma "receita" reutilizável que guarda um conjunto de definições para lançar instâncias — qual AMI usar, tipo de instância, VPC/subnet, security group, storage, key pair, etc. Serve para lançar novas instâncias mais depressa e de forma consistente, sem teres de preencher tudo manualmente de cada vez (muito usado, por exemplo, em Auto Scaling Groups)

A diferença chave: a AMI é o conteúdo (o disco, o SO, os ficheiros); o Launch Template é a configuração de como lançar uma instância (que geralmente referencia uma AMI, mas também define tipo de instância, rede, etc.). Não é que o Launch Template "copie o SO" — ele diz qual AMI usar, entre outras definições.

■ Progresso: 7h28min de vídeo assistido.

Aula 8

Aula 8 — Auto Scaling, Pricing Models e Segurança de Identidade

Aula 8 — Auto Scaling, Pricing Models e Segurança de Identidade

■■ PARTE 1 — SCALING E LOAD BALANCING

Auto Scaling Group (ASG) — passo a passo

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    A["1. Name"] --> B["2. Launch Template"]
    B --> C["3. VPC, AZ e Subnets"]
    C --> D["4. Load Balancer<br/>(opcional)"]
    D --> E["5. Group size<br/>Desired / Min / Max capacity"]
    E --> F["6. Scaling policies<br/>(métrica: CPU, rede, request count)"]
    F --> G["7. Notifications<br/>(opcional)"]
    G --> H["8. Tags<br/>(opcional)"]
```

Campo	O que define
Launch Template	A "receita" usada para lançar cada nova instância (AMI, tipo, security group, etc.)
VPC / AZ / Subnets	Onde as instâncias vão ser lançadas
Group size	Desired (quantas queres agora), Minimum e Maximum capacity
Scaling policies	Regra que decide quando adicionar/remover instâncias, baseada numa métrica: uso de CPU, uso de rede, ou request count per target (via Application Load Balancer)

Health checks

O ASG verifica continuamente a saúde de cada servidor. Se uma instância falhar o health check:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    HC["❤■ Health Check falha"] --> KILL["ASG termina a instância doente"]
    KILL --> NEW["ASG lança uma nova instância"]
    NEW --> ROUTE["Tráfego é reencaminhado<br/>para a nova instância saudável"]
```

Elastic Load Balancer (ELB) — tipos

Tipo	Quando usar
Application Load Balancer (ALB)	Conjunto flexível de funcionalidades para a tua app, tráfego HTTP/HTTPS
Network Load Balancer (NLB)	Quando precisas de alta performance, TLS offloading em escala, deployment centralizado de certificados, suporte a UDP e IP estático
Gateway Load Balancer (GWLB)	Quando precisas de implementar e gerir uma frota de appliances virtuais de terceiros que suportam o protocolo GENEVE

Configurar um Application Load Balancer

Campo	Detalhe
Name	Nome do ALB
Scheme	Internet-facing (público) ou Internal (só dentro da VPC)
IPv4	Endereçamento IP
VPC	A mesma VPC das instâncias
AZ	■■ Escolher as mesmas AZs usadas no Auto Scaling Group
Security Group	Firewall do próprio load balancer
Listeners and routing	Define o target group — as máquinas (ou IPs estáticos) para onde o tráfego é encaminhado

Depois de criado, o ALB é associado ao ASG, passando a distribuir o tráfego automaticamente pelas instâncias que o grupo gerir.

■ PARTE 2 — MODELOS DE PRICING DO EC2

Visão geral

Modelo	Ideal para	Compromisso	Poupança
On-Demand	Curto prazo, picos, imprevisível, não pode ser interrompido	Nenhum	Menor
Spot	Jobs não-críticos, em background, tolerantes a interrupções	Nenhum, mas pode ser interrompido pela AWS	Maior (até 90%)
Reserved (RI)	Uso estável e previsível a longo prazo	1 ou 3 anos	Alta a longo prazo
Dedicated	Precisas de hardware fisicamente isolado	Pode ser On-Demand, Reserved ou Spot	O mais caro
Savings Plans	Como o Reserved, mas mais simples e flexível	1 ou 3 anos de compromisso de gasto (não de instância fixa)	Alta, com mais flexibilidade

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["■ Mais caro<br/>Dedicated"] --> B["On-Demand"]
    B --> C["Savings Plans"]
    C --> D["Reserved Instances"]
    D --> E["■ Mais barato<br/>Spot"]
```

On-Demand

Modelo pay-as-you-go: consomes compute e depois pagas.

- Sem pagamento adiantado, sem compromisso.
- Usa para workloads de curto prazo, com picos ou imprevisíveis.

Reserved Instances (RI)

Desenhadas para aplicações com uso estável e previsível, ou que precisam de capacidade reservada.

“O desconto no preço depende de: Term × Class × Atributos da RI × Opção de pagamento”

Term (prazo de compromisso)

- Quantos anos de compromisso? Mais anos = mais poupança a longo prazo.
- ■■ Quando expira, é preciso renovar. Se não renovares, as instâncias passam a On-Demand, sem qualquer interrupção de serviço.

Class (classe)

Classe	Poupança	Flexibilidade
Standard	Até 75% de redução de preço	Pode modificar atributos
Convertible	Até 54% de redução de preço	Pode ser trocada (exchange) por outra RI, se o valor for igual ou superior

"■ Regra geral: quanto menos flexível, maior a poupança — a Standard poupa mais, mas é mais rígida; a Convertible poupa menos, mas adapta-se a mudanças."

Opções de pagamento

Opção	Poupança
All Upfront	Maior poupança
Partial Upfront	Poupança intermédia
No Upfront	Menor poupança

"■ Regra geral: quanto maior o valor pago adiantado, maior a poupança."

- As RIs podem ser partilhadas entre várias contas dentro de uma AWS Organization.
- RIs não usadas podem ser vendidas no Reserved Instance Marketplace.

Atributos de uma RI

- Instance type
- Region
- Tenancy
- Platform (Windows ou Linux)

Regional RI vs. Zonal RI

"Ao comprares uma RI, defines o scope (âmbito). O scope não afeta o preço."

	Regional RI	Zonal RI
Comprada para	Uma região	Uma Availability Zone específica
Reserva capacidade?	■ Não reserva capacidade	■ Reserva capacidade na AZ especificada
Onde se aplica o desconto	Em qualquer AZ da região	Só na AZ selecionada (sem flexibilidade de AZ)
Flexibilidade de tamanho	Aplica-se à utilização dentro da família de instância, independentemente do tamanho (só em Amazon Linux/Unix, tenancy default)	Sem flexibilidade de tamanho — o desconto só se aplica ao tipo e tamanho exatos especificados
Compras em fila (queued)	■ Permite colocar compras em fila	—

"■ Padrão para memorizar: Regional = flexível, mas não garante capacidade. Zonal = garante capacidade, mas é rígida."

Limites de compra de RI

Âmbito	Limite por mês
Regional RI	20 por Região
Zonal RI	20 por AZ

Tipo de limite	Regra
Regional	Não podes exceder o teu limite de instâncias On-Demand em execução, comprando RIs regionais. O limite por defeito de instâncias On-Demand é 20. Antes de comprar RIs, confirma que o teu limite On-Demand é igual ou superior ao número de RIs que pretendes comprar
Zonal	Podes exceder o teu limite de instâncias On-Demand comprando RIs zonais. Exemplo: se já tens 20 instâncias On-Demand em execução e compras 20 RIs zonais, podes lançar mais 20 instâncias On-Demand que correspondam às especificações dessas RIs zonais

Standard RI vs. Convertible RI

	Standard RI	Convertible RI
Modificar atributos	■ Sim — pode alterar a AZ dentro da mesma região, mudar o scope (Zonal ↔ Regional), mudar o tamanho da instância (só Linux/Unix, tenancy default), mudar a rede (EC2-Classic ↔ VPC)	■ Não pode ser modificada — em vez disso, faz-se uma troca (exchange)
Exchange (troca)	■ Não pode ser trocada	■ Pode ser trocada durante o termo por outra Convertible RI com novos atributos: família de instância, tipo, plataforma, scope, tenancy
RI Marketplace	■ Pode ser comprada/vendida	■ Não pode ser comprada/vendida

RI Marketplace — regras principais

- Só podes vender uma RI depois de estar ativa durante, no mínimo, 30 dias, e depois de a AWS ter recebido o pagamento adiantado (se aplicável).
- É preciso ter uma conta bancária dos EUA para vender RIs no marketplace.
- Tem de restar pelo menos 1 mês no termo da RI que estás a vender.
- Mantens o preço e o benefício de capacidade da reserva até à venda ser concluída.
- O nome da tua empresa (e morada, se pedido) é partilhado com o comprador, para fins fiscais.
- Só podes definir o preço adiantado (upfront) — o preço de utilização e a restante configuração (tipo de instância, AZ, plataforma) mantêm-se iguais à compra original.
- A duração do termo é arredondada para baixo ao mês mais próximo (ex.: 9 meses e 15 dias aparece como 9 meses).
- Podes vender até $20.000 em RIs por ano. Para vender mais, é preciso pedir um aumento.
- RIs na região GovCloud não podem ser vendidas no marketplace.

Capacity Reservations

As instâncias EC2 correm em hardware físico finito — por vezes, ao tentares lançar um tipo específico de instância, a AWS pode não ter capacidade disponível nesse momento.

“Capacity Reservations: serviço que te permite reservar capacidade de EC2 para uma região e AZ específicas, garantindo que o hardware estará disponível quando precisares.”

Spot Instances

Capacidade de computação não utilizada, que a AWS quer rentabilizar (como um avião a oferecer descontos para preencher lugares vagos).

- Até 90% de desconto face ao preço On-Demand.
- Desenhadas para apps com horários de início/fim flexíveis, ou que só são viáveis a custos de computação muito baixos.
- AWS Batch é uma forma fácil e conveniente de usar preços Spot.

Situação	É cobrado?
A AWS termina a instância	■ Não pagas a hora parcial de utilização
Tu terminas a instância	■ Continuas a ser cobrado

"■■ As instâncias Spot podem ser terminadas pela AWS a qualquer momento — nunca uses para workloads críticos."

Dedicated Instances e Dedicated Hosts

Úteis quando precisas de cumprir requisitos regulatórios, ou quando tens licenciamento de software ligado a hardware físico específico que não suporta multi-tenancy nem deployments em cloud partilhada.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph MULTI["Multi-tenancy<br/>'viver num prédio'"]
        M1["Cliente A"] --- SRV1["Mesmo servidor físico"]
        M2["Cliente B"] --- SRV1
        M3["Cliente C"] --- SRV1
    end
    subgraph SINGLE["Single-tenant<br/>'ter casa própria'"]
        S1["Cliente único"] --- SRV2["Servidor físico dedicado"]
    end
```

Modelo	Isolamento	Analogia
Multi-tenancy	Virtual — vários clientes correm workloads no mesmo servidor físico	Viver num prédio: vários inquilinos, paredes a separar
Single-tenant	Físico — um servidor dedicado a um único cliente	Ter casa própria: só tu nesse edifício

"A tenancy escolhe-se no momento em que lanças a instância EC2."

Savings Plans

Descontos semelhantes às Reserved Instances, mas com um processo de compra mais simples: em vez de reservares um tipo de instância específico, comprometes-te a um valor de gasto por hora.

Tipo	Poupança	Flexibilidade
Compute Savings Plans	Até 66%	Máxima flexibilidade — aplica-se automaticamente a EC2, AWS Fargate e AWS Lambda, independentemente da família de instância, tamanho, AZ, SO ou tenancy
EC2 Instance Savings Plans	Até 72%	Preços mais baixos, mas o compromisso é para o uso de uma família de instância específica
SageMaker Savings Plans	Até 64%	Aplica-se automaticamente ao uso do SageMaker, independentemente da família de instância, tamanho, componente ou região

Termos e pagamento: iguais às RIs — 1 ou 3 anos, com opções All Upfront / Partial Upfront / No Upfront.

■ PARTE 3 — ZERO TRUST E GESTÃO DE IDENTIDADE

O que é o Zero Trust Model

“Princípio: "não confies em ninguém, verifica tudo" ("trust no one, verify everything").”

A identidade passa a ser o principal perímetro de segurança — em vez de proteger "a rede" como um todo, protege-se e controla-se o acesso aos dados e recursos da empresa, verificando sempre quem está a pedir acesso, independentemente de onde vem o pedido.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph OLD["■ Modelo tradicional (network-centric)"]
        N1["Dentro da rede corporativa = confiável"]
        N2["Fora da rede = não confiável"]
    end
    subgraph ZT["■ Zero Trust (identity-centric)"]
        Z1["Cada pedido é verificado,<br/>independentemente da origem"]
        Z2["A identidade é o novo perímetro"]
    end
```

Abordagem	Onde está o "perímetro"
Network-centric (tradicional)	A rede — se estás "dentro", és confiável
Identity-centric (Zero Trust)	A identidade — cada pedido é verificado, mesmo vindo de dentro

Zero Trust na AWS

Serviço	Papel no Zero Trust
AWS IAM	IAM policies, permissions boundaries, service control policies (SCPs), e condições em políticas IAM (restringir por IP, região, se o MFA está desligado, hora do dia)
AWS CloudTrail	Regista (audita) todas as chamadas à API feitas na conta
Amazon GuardDuty	Deteta atividade suspeita ou maliciosa, com base nos logs do CloudTrail e outras fontes
Amazon Detective	Analisa, investiga e identifica rapidamente problemas de segurança — pode consumir os findings do GuardDuty

“■■ Nota importante: a AWS não tem, de raiz, controlos de identidade "inteligentes" prontos a usar — por isso considera-se que a AWS não tem, por si só, uma verdadeira oferta Zero Trust completa. Normalmente são usados serviços e identity providers de terceiros em conjunto com a AWS para atingir um modelo Zero Trust completo.”

Zero Trust na AWS com terceiros

O Azure Active Directory (e outros, como o Google BeyondCorp ou o JumpCloud) consegue deteção de risco em tempo real e calculada, com base em mais pontos de dados do que a AWS sozinha: dispositivo e app usados, hora do dia, localização, se o MFA está ativo, e o que está a ser acedido.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    AAD["Azure AD<br/>(ou Google BeyondCorp, JumpCloud, ...)"] --> SSO["AWS Single Sign-On"]
    SSO --> RES["Recursos AWS"]
```

Directory Service

Componente crítico de um sistema operativo de rede: mapeia nomes de recursos de rede para os respetivos endereços de rede.

“É uma infraestrutura de informação partilhada para localizar, gerir, administrar e organizar recursos: volumes, pastas, ficheiros, impressoras, utilizadores, grupos, dispositivos, etc.”

Exemplo	Descrição
DNS	O "directory service" da internet
Microsoft Active Directory / Azure Active Directory	Directory service corporativo da Microsoft
Apache Directory Server	Implementação open-source
OpenLDAP	Implementação open-source do protocolo LDAP

Active Directory

A Microsoft introduziu o Active Directory Domain Services no Windows 2000: permite gerir múltiplos componentes e sistemas de infraestrutura on-premise, usando uma única identidade por utilizador.

Identity Providers (IdPs)

“Entidade de sistema que cria, mantém e gere informação de identidade de principais (utilizadores), e também fornece serviços de autenticação a aplicações dentro de uma federação ou rede distribuída.”

Exemplos: Facebook, Amazon, Google, Twitter, GitHub, LinkedIn.

Federated Identity (Identidade Federada)

Método que permite ligar a identidade de um utilizador entre múltiplos sistemas de gestão de identidade separados — ou seja, usar a mesma conta (ex.: Google) para entrar em vários sistemas diferentes.

Protocolos de autenticação/federação

Protocolo	Tipo	Como funciona
OpenID	Protocolo de autenticação	Permite fazer login numa plataforma diferente usando, por exemplo, uma conta Google
OAuth 2.0	Protocolo de autorização	Não partilha dados de password — usa tokens de autorização para provar uma identidade entre o consumidor e o fornecedor do serviço
SAML	Protocolo de Single Sign-On	SSO feito via browser web

Single Sign-On (SSO)

Permite que um utilizador entre com um único ID e password em vários sistemas e aplicações diferentes.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    AD["Azure AD"] -->|SAML| SSO["Single Sign-On"]
    SSO --> S1["Slack"]
    SSO --> S2["AWS"]
    SSO --> S3["Google"]
    SSO --> S4["Salesforce"]
    SSO --> S5["PC"]
```

“Fazes login no teu diretório principal uma única vez, e consegues entrar em múltiplas plataformas sem introduzir as credenciais novamente em cada uma.”

LDAP (Lightweight Directory Access Protocol)

Protocolo de aplicação para aceder e manter serviços de informação de diretório distribuída, via Internet Protocol (IP).

- Permite o "mesmo" sign-on (same sign-on): o utilizador usa um único ID e password, mas tem de o introduzir de cada vez que quer entrar (ao contrário do SSO verdadeiro, onde só entras uma vez).

```
on-premise AD → LDAP → Google, Kubernetes, Jenkins
```

“■ Exemplo prático dado na aula: usado numa firewall empresarial — para ligar à VPN, configurava-se o utilizador no portal da Fortinet, usando o mesmo login/password para entrar na VPN.”

SSO vs. LDAP — a diferença chave

	SSO	LDAP (same sign-on)
Nº de vezes que introduzes a password	Uma vez por sessão, para todos os sistemas	De cada vez que entras num sistema, embora a password seja sempre a mesma
Experiência do utilizador	Mais fluida — não vês ecrã de login repetido	Vês sempre o ecrã de login, mas usas sempre as mesmas credenciais

MFA (Multi-Factor Authentication)

Controlo de segurança em que, além de username/email e password, tens de usar um segundo dispositivo (ex.: telemóvel) para confirmar que és mesmo tu.

- Protege contra quem roubou a tua password, mas não tem o segundo fator.
- Exemplo: Google Authenticator.
- ■■ É muito importante ativar MFA em todas as contas AWS, especialmente na conta root.

Security Keys

Dispositivo secundário usado como segundo passo no processo de autenticação, para aceder a um dispositivo, app ou posto de trabalho.

“Exemplo: YubiKey — chave física onde colocas o dedo (ou inseres numa porta USB/NFC) para autenticar.”

■ Progresso: 8h16min de vídeo assistido.

Aula 9

Aula 9 — IAM Avançado, Application Integration, Containers e Governança

Aula 9 — IAM Avançado, Application Integration, Containers e Governança

■ PARTE 1 — IAM EM DETALHE

As três identidades do IAM

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    IAM["AWS IAM"] --> U["■ IAM Users<br/>utilizadores finais que fazem<br/>login na consola ou API"]
    IAM --> G["■ IAM Groups<br/>agrupam users que partilham<br/>o mesmo nível de permissões"]
    IAM --> R["■ IAM Roles<br/>dão permissões a RECURSOS AWS<br/>para executar ações da API"]
```

Identidade	O que é	Exemplo
IAM User	Utilizador final que entra na consola ou interage com recursos AWS	A tua própria conta de trabalho
IAM Group	Conjunto de users que partilham o mesmo nível de permissões	Grupo "Developers" com acesso a EC2 e S3
IAM Role	Dá a um recurso AWS (não a uma pessoa) permissão para executar ações específicas da API. Associas uma policy a uma role e atribuis essa role ao recurso	Uma EC2 com uma role que lhe permite ler de um bucket S3 específico

Anatomia de uma IAM Policy

- IAM Policies: documentos JSON que concedem permissões a um user, group ou role específicos, para acederem a serviços. São anexadas a identidades IAM.
- IAM Permissions: as ações da API que podem (ou não) ser executadas — são representadas dentro do documento da policy.

“Uma policy é, no fundo, uma lista de "o que esta identidade pode ou não fazer", escrita em JSON.”

Princípio do Menor Privilégio (Least Privilege)

Conceito de segurança informática: atribuir a um user, role ou app apenas as permissões mínimas necessárias para executar a sua função.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["Just Enough Access<br/>permitir só as ações exatas<br/>necessárias à identidade"]
    B["Just In Time<br/>permitir o acesso só durante<br/>o período mínimo necessário"]
```

Conceito	Significado
Just Enough Access	Permitir apenas as ações exatas que a identidade precisa — nada a mais
Just In Time	Permitir o acesso apenas durante o tempo mínimo necessário, não permanentemente
Risk-Based Adaptive Policies	Cada tentativa de acesso a um recurso gera uma pontuação de risco, com base em fatores como dispositivo, localização, IP, serviço acedido e horário

“■■ Importante: a AWS não tem políticas adaptativas baseadas em risco incorporadas de raiz no IAM — este tipo de funcionalidade normalmente vem de ferramentas de terceiros (ligação direta ao que vimos na Aula 8 sobre Zero Trust).”

Conta Root vs. Utilizador IAM

	AWS Account Root User	AWS Account User (IAM)
Acesso	Total, a todos os serviços e recursos — não pode ser limitado por policies IAM	Definido pelas policies IAM atribuídas
Quantidade	Só pode haver um por conta AWS; não pode ser apagado	Tantos quantos forem necessários
Login	Email + password	Account ID + username + password

Ações que só a conta root pode executar

- Alterar definições da conta: nome, email, password do root
- Restaurar permissões de um utilizador IAM
- Ativar acesso IAM à consola de Billing and Cost Management
- Ver determinadas faturas fiscais
- Fechar a conta AWS
- Alterar ou cancelar o plano de suporte AWS
- Ativar o MFA Delete num bucket S3

“■ Recomendação forte: nunca usar access keys da conta root, e ativar sempre MFA no root.”

AWS IAM Identity Center

“Antigo AWS Single Sign-On: crias ou ligas as identidades da tua força de trabalho à AWS uma vez, e geres o acesso de forma centralizada em toda a tua AWS Organization.”

■ PARTE 2 — APPLICATION INTEGRATION

Application Integration: processo de permitir que duas aplicações independentes comuniquem e trabalhem em conjunto.

Os três padrões de comunicação

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    Q["■ Queueing (SQS)<br/>mensagens apagadas após consumo<br/>não é tempo real, é preciso fazer pull"]
    S["■ Streaming (Kinesis)<br/>mensagens ficam disponíveis por mais tempo<br/>múltiplos consumidores, tempo real"]
    P["■ Pub/Sub (SNS)<br/>mensagens enviadas (push) automaticamente<br/>a quem subscreveu"]
```

Padrão	Como funciona	Serviço AWS
Queueing	Sistema de mensagens simples; as mensagens são apagadas depois de consumidas. Não é tempo real — é preciso fazer pull (ir buscar)	Amazon SQS
Streaming	As mensagens/eventos ficam disponíveis na stream durante mais tempo, permitindo operações complexas. Múltiplos consumidores podem reagir. Tempo real	Amazon Kinesis

Padrão	Como funciona	Serviço AWS
Pub/Sub	O publicador não sabe quem são os subscritores. As mensagens são automaticamente enviadas (push) a quem subscreveu — não há pull	Amazon SNS

Amazon SQS (Simple Queue Service)

Serviço de queueing totalmente gerido, que permite desacoplar e escalar microservices, sistemas distribuídos e aplicações serverless.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    APP1["App remetente"] -->|envia mensagem| Q["■ SQS Queue"]
    Q -->|pull| APP2["App consumidora"]
```

Amazon Kinesis

Solução totalmente gerida para recolher, processar e analisar dados em streaming na cloud.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    P1["EC2"] --> K["Kinesis Data Streams"]
    P2["Mobile"] --> K
    P3["Client"] --> K
    P4["Servidor on-prem"] --> K
    K --> S1["Shard 1"]
    K --> S2["Shard 2"]
    K --> S3["Shard n"]
    S1 --> C1["Redshift"]
    S2 --> C2["DynamoDB"]
    S3 --> C3["S3"]
    S1 --> C4["EMR"]
```

Amazon SNS (Simple Notification Service)

Serviço de mensagens pub/sub da AWS.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    PR1["AWS SDK"] --> SNS["SNS"]
    PR2["AWS CLI"] --> SNS
    PR3["CloudWatch"] --> SNS
    PR4["Serviços AWS"] --> SNS
    SNS --> TOPIC["SNS Topic"]
    TOPIC -->|"filtragem + fanout"| SUB1["Lambda"]
    TOPIC --> SUB2["SQS"]
    TOPIC --> SUB3["Email"]
    TOPIC --> SUB4["HTTP/S"]
```

“Formatos de entrega suportados: texto simples, email, HTTP (webhooks), SMS (mensagens de texto), SQS e Lambda.”

Amazon API Gateway

Programa que se posiciona entre um único ponto de entrada e múltiplos backends — permite throttling, logging, lógica de routing, e formatação de pedidos/respostas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    APP1["App móvel"] --> GW["API Gateway"]
```

```
APP2["App web"] --> GW
APP3["Dispositivo IoT"] --> GW
GW --> CACHE["API Gateway Cache"]
GW --> CW["CloudWatch"]
GW --> SVC["Serviços AWS<br/>(EC2, Lambda, RDS, ...)"]
```

“Amazon API Gateway: cria APIs seguras no teu ambiente cloud, a qualquer escala.”

Máquinas de Estado e AWS Step Functions

State machine: decide como se passa de um estado para outro, com base numa série de condições — como um fluxograma.

“AWS Step Functions: coordena múltiplos serviços AWS num workflow serverless. Tem uma consola gráfica para visualizar os componentes, aciona e acompanha automaticamente cada passo, faz retry em caso de erro, e regista (log) o estado de cada passo.”

Event Bus e Amazon EventBridge

Event bus: recebe eventos e encaminha-os para um alvo (target), com base em regras.

“Amazon EventBridge (antigo CloudWatch Events): serviço de event bus serverless, usado para integração de aplicações através de streaming de dados em tempo real.”

Resumo: Serviços de Application Integration

Serviço	Tipo	Descrição
SNS	Pub/Sub	Vários formatos de entrega: texto, email, HTTP (webhooks), SMS, SQS, Lambda
SQS	Queueing	Envia eventos para uma fila; outras apps fazem pull da fila para obter mensagens
Step Functions	State machine	Serviço de máquina de estados para orquestrar workflows
EventBridge	Event bus	Event bus serverless
Kinesis	Streaming	Dados em streaming em tempo real, com múltiplos consumidores
Amazon MQ	Message broker	Serviço de message broker gerido, usa Apache ActiveMQ
Managed Kafka (MSK)	Message broker	Apache Kafka totalmente gerido
API Gateway	API management	Criar, publicar, manter, monitorizar e proteger APIs
AppSync	GraphQL	Serviço GraphQL totalmente gerido

AWS Amplify

Framework opinativo e infraestrutura totalmente gerida, para os developers se focarem em construir apps web e mobile.

Componente	Função
Amplify CLI	Linha de comandos para provisionar recursos
Amplify SDK	Biblioteca para integrar a app com os serviços AWS
Amplify UI	Componentes de interface prontos a usar
Amplify Hosting	Hospedagem da aplicação

Componente	Função
Amplify Studio	Interface visual para gerir o backend

“Funciona com frameworks JavaScript.”

AWS AppSync

Serviço GraphQL totalmente gerido.

Conceito	Descrição
Schema	Representa o teu modelo de dados
Data sources	De onde os dados são obtidos: DynamoDB, Amazon OpenSearch, AWS Lambda, endpoint HTTP, etc.
Resolvers	Anexados a campos específicos dentro dos teus tipos no schema — implementam as operações de alteração de estado para query, mutation e subscription

- Suporta APIs em tempo real via GraphQL subscriptions.

Tipo de API	Descrição
GraphQL API	Uma única API a partir de múltiplas fontes de dados
Merged API	Coleção de GraphQL APIs que funcionam como uma só — útil quando várias equipas gerem as suas próprias APIs, mas queres tratá-las como uma API única

Opções de caching: nenhuma / caching completo do pedido / caching por resolver (operações ou campos específicos).

■ PARTE 3 — VMs vs. Containers (explicado em detalhe)

O problema das VMs

Numa máquina virtual, cada app corre isolada num SO convidado completo, por cima de um hypervisor. Isto tem um custo:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph VM["Virtual Machine"]
        OS["SO convidado (ex: Ubuntu)"]
        APP1["Django"]
        APP2["MongoDB"]
        APP3["RabbitMQ"]
        LIB["Libraries, Packages, Binaries"]
        WASTE["■ Espaço desperdiçado"]
    end
    HYP["Hypervisor"] --> VM
    HOST["Host Operating System"] --> HYP
```

- As VMs não fazem o melhor uso do espaço.
- As apps não estão isoladas entre si dentro da mesma VM — podem gerar conflitos de configuração, problemas de segurança ou monopolização de recursos (resource hogging) por uma app em detrimento das outras.

A solução: Containers

Um container empacota a aplicação com tudo o que ela precisa para correr (bibliotecas, dependências), mas partilha o kernel do sistema operativo do host, em vez de ter o seu próprio SO completo.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph C["EC2 Instance com Docker"]
        subgraph CT1["Container 1"]
            D["Django"]
            L1["Lib/Pkg/Bin"]
        end
        subgraph CT2["Container 2"]
            M["MongoDB"]
            L2["Lib/Pkg/Bin"]
        end
        subgraph CT3["Container 3"]
            R["RabbitMQ"]
            L3["Lib/Pkg/Bin"]
        end
        FREE["■ Espaço disponível<br/>para mais containers"]
    end
    DAEMON["Docker Daemon"] --> C
    HOST2["Host Operating System"] --> DAEMON
```

- Os containers permitem correr múltiplas aplicações, virtualmente isoladas umas das outras.
- Podes lançar novos containers e configurar as dependências do SO por container, sem desperdiçar o espaço que sobra, como acontecia com as VMs.

Comparação direta

	Máquina Virtual (VM)	Container
Camada de virtualização	Hypervisor	Container Engine (ex.: Docker Daemon)
O que cada unidade tem	O seu próprio SO completo	Partilha o kernel do SO do host
Peso/arranque	Mais pesado, arranque mais lento (minutos)	Muito mais leve, arranque quase instantâneo (segundos)
Isolamento	Isolamento forte (ao nível do SO)	Isolamento mais leve (ao nível de processo)
Uso de espaço	Tende a desperdiçar recursos reservados e não usados	Aproveita melhor o espaço disponível na mesma instância
Melhor para	Correr sistemas operativos completamente diferentes na mesma máquina física	Empacotar e distribuir aplicações de forma consistente e portátil

“■ Resumo mental: VM = virtualizas o hardware (cada VM pensa que tem o seu próprio computador). Container = virtualizas o sistema operativo (cada container pensa que tem o seu próprio SO, mas todos partilham o mesmo kernel por baixo).”

■ PARTE 4 — Microservices (explicado em detalhe)

Arquitetura Monolítica vs. Microservices

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph MONO["■ Monolithic Architecture<br/>UMA app responsável por tudo<br/>Funcionalidade fortemente acoplada"]
        direction TB
        S1["Services: Load Balancing, Caching, Database"]
        S2["Marketing: HTML/CSS, JavaScript, Assets"]
        S3["Frontend: JS Framework, HTML Templates"]
        S4["Backend: API, ORM, PDFs, Reports, Thumbnails"]
    end
```

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph MICRO["■■ Microservices Architecture<br/>Múltiplas apps, cada uma responsável por UMA coisa<br/>Isoladas e stateles
        direction TB
```

```
        M1["Load Balancing"]
        M2["Caching"]
        M3["Database"]
        M4["Queueing"]
        M5["HTML/CSS"]
        M6["JavaScript"]
        M7["Assets"]
        M8["API"]
        M9["ORM"]
        M10["Reports"]
        M11["■■ PDFs (Lambda)"]
        M12["■■ Thumbnails (Lambda)"]
    end
```

	Monolito	Microservices
Nº de aplicações	Uma só, responsável por tudo	Várias, cada uma responsável por uma coisa
Acoplamento	Fortemente acoplado — tudo depende de tudo	Isolado e stateless — cada serviço é independente
Deploy	Deploy de tudo de uma vez, mesmo para uma mudança pequena	Deploy independente de cada serviço
Escalar	Escalas a aplicação inteira, mesmo que só uma parte precise	Escalas só o serviço que precisa (ex.: só o serviço de "Thumbnails")
Falha	Uma falha pode derrubar toda a aplicação	Uma falha fica contida nesse serviço específico
Tecnologia	Normalmente uma stack única para tudo	Cada serviço pode usar a tecnologia mais adequada (ex.: PDFs e Thumbnails como funções Lambda)
Complexidade operacional	Mais simples de gerir no início	Mais complexa — precisas de orquestração, service discovery, monitorização distribuída

Vantagens dos microservices: escalabilidade seletiva, resiliência a falhas localizadas, equipas podem trabalhar e fazer deploy independentemente em cada serviço, liberdade tecnológica por serviço.

Desvantagens: maior complexidade de infraestrutura (rede, comunicação entre serviços), mais difícil de testar de ponta a ponta, exige ferramentas de orquestração (é aqui que entra o Kubernetes, a seguir).

■■ PARTE 5 — Kubernetes (K8s)

Sistema de orquestração de containers open-source, para automatizar o deployment, scaling e gestão de containers.

- Criado originalmente pela Google, atualmente mantido pela Cloud Native Computing Foundation (CNCF).
- Chamado habitualmente de "K8s": o "8" representa as letras restantes de "ubernetes".
- Vantagem sobre o Docker sozinho: permite correr containers distribuídos por várias VMs.

Arquitetura do Kubernetes

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph MASTER["Kubernetes Master"]
        SCHED["Scheduler"]
        CTRL["Controllers"]
        ETCD["Etcd"]
    end
    API["API Server"]
    MASTER --> API

    subgraph NODE1["Node"]
        KL1["Kubelet"]
        PX1["Proxy"]
        subgraph POD1["Pod"]
            C1["Container"]
            C2["Container"]
```

```
        end
        subgraph POD2["Pod"]
            C3["Container"]
            C4["Container"]
        end
    end

    subgraph NODE2["Node"]
        KL2["Kubelet"]
        PX2["Proxy"]
        subgraph POD3["Pod"]
            C5["Container"]
            C6["Container"]
        end
        subgraph POD4["Pod"]
            C7["Container"]
            C8["Container"]
        end
    end

    API --> NODE1
    API --> NODE2
```

Componente	Função
Scheduler	Decide em que node cada pod vai correr
Controllers	Garantem que o estado atual do cluster corresponde ao estado desejado
Etcd	Base de dados chave-valor que guarda o estado do cluster
API Server	Ponto de entrada para todos os comandos ao cluster
Node	Uma máquina (VM ou física) que corre pods
Kubelet	Agente que corre em cada node, garante que os containers dos pods estão a correr
Proxy	Gere as regras de rede em cada node
Pod	Grupo de um ou mais containers com storage, recursos de rede e outras configurações partilhadas — é a unidade mais pequena que se pode gerir no Kubernetes

"■ O Kubernetes é ideal para arquiteturas de microservices, onde uma empresa tem dezenas a centenas de serviços para gerir — exatamente o cenário descrito na Parte 4."

■ PARTE 6 — Docker e Podman

Docker

Produto que usa virtualização ao nível do SO para entregar software em pacotes chamados containers.

Componente	Função
Docker CLI	Descarregar, enviar, construir, correr e debugar containers
Dockerfile	Ficheiro de configuração com as instruções para provisionar um container
Docker Compose	Ferramenta + ficheiro de configuração para trabalhar com múltiplos containers
Docker Swarm	Ferramenta de orquestração para gerir arquiteturas de múltiplos containers em produção
Docker Hub	Repositório online público de containers, publicados pela comunidade, para download

Podman

Motor de containers compatível com OCI (Open Container Initiative), pensado como substituto direto ("drop-in replacement") do Docker.

Diferença	Docker	Podman
Arquitetura	Usa um daemon (containerd)	Sem daemon
Suporte a Pods	Não	Sim — tal como o Kubernetes

“O Podman substitui apenas parte do que o Docker faz; costuma usar-se em conjunto com:Buildah — para construir imagens OCISkopeo — para mover imagens de container entre diferentes tipos de storage”

Serviços de Containers na AWS

Categoria	Serviços
Primários (correm os containers)	ECS, AWS Fargate, EKS, AWS Lambda
Provisioning e deployment	Elastic Beanstalk (EB), App Runner, AWS Copilot CLI
Serviços de suporte	Elastic Container Registry (imagens Docker), X-Ray, Step Functions

■ PARTE 7 — Organizations, Control Tower e Governança

AWS Organizations

Permite criar novas contas AWS e gerir centralmente billing, controlo de acesso, compliance, segurança e partilha de recursos entre contas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    MASTER["■ Master Account<br/>(The Federation)"]
    MASTER --> OU1["■■ Organization Unit:<br/>Starfleet"]
    MASTER --> OU2["■■ Organization Unit:<br/>Federation Planets"]

    OU1 --> A1["Conta: Section 31"]
    OU1 --> A2["Conta: Corps of Engineers"]
    OU1 --> A3["Conta: Starfleet Command"]

    OU2 --> B1["Conta: Earth"]
    OU2 --> B2["Conta: Vulcan"]
    OU2 --> B3["Conta: Betazed"]
```

Conceito	Descrição
Root Account User	Identidade de login única, com acesso completo a todos os serviços e recursos numa conta. Cada conta tem o seu Root Account User
Organization Units (OUs)	Grupos de contas AWS dentro de uma organização — podem conter outras OUs, criando uma hierarquia
Service Control Policies (SCPs)	Dão controlo central sobre as permissões máximas permitidas em todas as contas da organização, ajudando a garantir que as contas se mantêm dentro das regras da organização

“■■ O AWS Organizations, depois de ativado, não pode ser desativado. Podes criar quantas contas AWS quiseres — uma delas será sempre a conta Master/Root da organização.”

AWS Control Tower

Ajuda empresas a montar rapidamente um ambiente multi-conta AWS seguro, fornecendo um ambiente de base (baseline) para começar com uma arquitetura multi-conta.

Componente	Descrição
Landing Zone	Ambiente de base: ativa o AWS SSO, logging centralizado via AWS CloudTrail, e auditoria de segurança entre contas
Account Factory	Automatiza o provisionamento de novas contas na organização, padronizando o processo
Guardrails	Regras de governança pré-empacotadas, para segurança, operações e compliance, que podes aplicar a toda a empresa ou a grupos específicos de contas

AWS Config

“Change management: ter um processo formal para monitorizar, aplicar e corrigir alterações. Compliance as Code: usar programação para automatizar a monitorização, aplicação e correção de alterações.”

AWS Config: framework de compliance as code, que permite gerir alterações numa base por região.

Quando usar o AWS Config?

- Queres que um recurso se mantenha configurado de uma forma específica, por motivos de compliance.
- Queres acompanhar alterações de configuração ao longo do tempo.
- Precisas de uma lista de todos os recursos existentes.
- Queres analisar potenciais fragilidades de segurança.

Como funciona

Defines regras para detetar situações específicas, e um plano de remediação — se a regra for violada, o Config pode corrigir automaticamente (ou notificar alguém para o fazer).

Exemplo — IAM Password Policy Conformance Pack:

- Comprimento mínimo da password
- Exigir letras minúsculas
- Exigir números
- Idade máxima da password
- Exigir prevenção de reutilização
- Exigir letras maiúsculas
- Exigir símbolos

Tagging (Etiquetagem)

Uma tag é um par chave-valor que atribuis a recursos AWS.

Exemplos de tags:

Key	Value
dept	it
status	approved
project	enterprise
location	london

Porque organizar recursos com tags

- Gestão de recursos
- Gestão e otimização de custos
- Gestão de operações
- Segurança
- Governança e compliance regulatório
- Automação
- Otimização de workloads

Resource Groups

Coleção de recursos que partilham uma ou mais tags. Aparecem no cabeçalho global da consola AWS, e também em Systems Manager.

■■ PARTE 8 — Cloud Adoption Framework (CAF)

Whitepaper da AWS para ajudar a migrar de on-premise para a AWS.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    CAF["AWS Cloud Adoption Framework"] --> B["Business"]
    CAF --> P["Platform"]
    CAF --> PE["People"]
    CAF --> S["Security"]
    CAF --> G["Governance"]
    CAF --> O["Operations"]
```

6 áreas de foco: Business, Platform, People, Security, Governance, Operations.

■ PARTE 9 — Serviços Orientados ao Negócio

Serviço	Função
Amazon Connect	Serviço de call center virtual
Amazon WorkSpaces	Serviço de ambiente de trabalho remoto virtual
Amazon WorkDocs	Serviço de colaboração partilhada (semelhante ao SharePoint)
Amazon Chime	Serviço de videoconferência (semelhante ao Zoom)
Amazon WorkMail	Serviço gerido de email empresarial, contactos e calendário
Amazon Pinpoint	Gestão de campanhas de marketing — envio de emails/SMS direcionados
Amazon SES (Simple Email Service)	Serviço de email transacional, integrável nas tuas aplicações
Amazon QuickSight	Serviço de Business Intelligence (BI)

■■ PARTE 10 — Serviços de Provisioning

“Provisioning: alocação ou criação de recursos e serviços para um cliente.”

Serviço	Função
Elastic Beanstalk	PaaS para fazer deploy de aplicações web facilmente
AWS OpsWorks	Serviço de configuration management, com instâncias geridas do Chef e Puppet (open-source)
AWS CloudFormation	Serviço de modelação e provisionamento de infraestrutura — automatiza o provisionamento ao escrever templates JSON/YAML (Infrastructure as Code)
AWS Quick Start	Pacotes pré-construídos para lançar e configurar rapidamente compute, rede, storage e outros serviços
AWS Marketplace	Catálogo digital de milhares de listagens de software de fornecedores independentes
AWS Amplify	Framework mobile e web que provisiona múltiplos serviços AWS como backend, com foco em serverless (DynamoDB, API Gateway, ...)
AWS App Runner	Facilita o deploy rápido de aplicações web e APIs containerizadas, a escala, sem experiência prévia de infraestrutura
AWS Copilot	CLI que permite lançar e gerir facilmente aplicações containerizadas na AWS
AWS CodeStar	Interface unificada para gerir atividades de desenvolvimento de software num único sítio
AWS CDK (Cloud Development Kit)	Ferramenta de IaC que permite usar a tua linguagem de programação preferida, gerando templates CloudFormation como mecanismo de IaC

■ Progresso: 9h45min de vídeo assistido.

Aula 10

Aula 10

aws elastic beansstalk

platform as service (paas): allows customers to develop, rin and manage app without completixy building and maintaining the infrastructure typically associated with developing and launching app

Elastic beanstalk: Paas for deploying web applications with little to no knowledge of the underlying infraestructure so you can focus on writting code insted of deployment pipeline and devops tasks. Not recommend for production app powered by cloudformation template setups for you: elastic load balancer, autoscalling groups, rds database, ec2 instance preconfigured, run dockerized environment

Anatomia de uma IAM Policy

- IAM Policies: documentos JSON que concedem permissões a um user, group ou role específicos, para acederem a serviços. São anexadas a identidades IAM.
- IAM Permissions: as ações da API que podem (ou não) ser executadas — são representadas dentro do documento da policy.

“Uma policy é, no fundo, uma lista de "o que esta identidade pode ou não fazer", escrita em JSON.”

Princípio do Menor Privilégio (Least Privilege)

Conceito de segurança informática: atribuir a um user, role ou app apenas as permissões mínimas necessárias para executar a sua função.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["Just Enough Access<br/>permitir só as ações exatas<br/>necessárias à identidade"]
    B["Just In Time<br/>permitir o acesso só durante<br/>o período mínimo necessário"]
```

Conceito	Significado
Just Enough Access	Permitir apenas as ações exatas que a identidade precisa — nada a mais
Just In Time	Permitir o acesso apenas durante o tempo mínimo necessário, não permanentemente
Risk-Based Adaptive Policies	Cada tentativa de acesso a um recurso gera uma pontuação de risco, com base em fatores como dispositivo, localização, IP, serviço acedido e horário

“■■ Importante: a AWS não tem políticas adaptativas baseadas em risco incorporadas de raiz no IAM — este tipo de funcionalidade normalmente vem de ferramentas de terceiros (ligação direta ao que vimos na Aula 8 sobre Zero Trust).”

Conta Root vs. Utilizador IAM

	AWS Account Root User	AWS Account User (IAM)
Acesso	Total, a todos os serviços e recursos — não pode ser limitado por policies IAM	Definido pelas policies IAM atribuídas
Quantidade	Só pode haver um por conta AWS; não pode ser apagado	Tantos quantos forem necessários
Login	Email + password	Account ID + username + password

Ações que só a conta root pode executar

- Alterar definições da conta: nome, email, password do root
- Restaurar permissões de um utilizador IAM
- Ativar acesso IAM à consola de Billing and Cost Management
- Ver determinadas faturas fiscais
- Fechar a conta AWS
- Alterar ou cancelar o plano de suporte AWS
- Ativar o MFA Delete num bucket S3

“■ Recomendação forte: nunca usar access keys da conta root, e ativar sempre MFA no root.”

AWS IAM Identity Center

“Antigo AWS Single Sign-On: crias ou ligas as identidades da tua força de trabalho à AWS uma vez, e geres o acesso de forma centralizada em toda a tua AWS Organization.”

■ PARTE 2 — APPLICATION INTEGRATION

Application Integration: processo de permitir que duas aplicações independentes comuniquem e trabalhem em conjunto.

Os três padrões de comunicação

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    Q["■ Queueing (SQS)<br/>mensagens apagadas após consumo<br/>não é tempo real, é preciso fazer pull"]
    S["■ Streaming (Kinesis)<br/>mensagens ficam disponíveis por mais tempo<br/>múltiplos consumidores, tempo real"]
    P["■ Pub/Sub (SNS)<br/>mensagens enviadas (push) automaticamente<br/>a quem subscreveu"]
```

Padrão	Como funciona	Serviço AWS
Queueing	Sistema de mensagens simples; as mensagens são apagadas depois de consumidas. Não é tempo real — é preciso fazer pull (ir buscar)	Amazon SQS
Streaming	As mensagens/eventos ficam disponíveis na stream durante mais tempo, permitindo operações complexas. Múltiplos consumidores podem reagir. Tempo real	Amazon Kinesis
Pub/Sub	O publicador não sabe quem são os subscritores. As mensagens são automaticamente enviadas (push) a quem subscreveu — não há pull	Amazon SNS

Amazon SQS (Simple Queue Service)

Serviço de queueing totalmente gerido, que permite desacoplar e escalar microservices, sistemas distribuídos e aplicações serverless.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    APP1["App remetente"] -->|envia mensagem| Q["■ SQS Queue"]
    Q -->|pull| APP2["App consumidora"]
```

Amazon Kinesis

Solução totalmente gerida para recolher, processar e analisar dados em streaming na cloud.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    P1["EC2"] --> K["Kinesis Data Streams"]
    P2["Mobile"] --> K
    P3["Client"] --> K
    P4["Servidor on-prem"] --> K
    K --> S1["Shard 1"]
    K --> S2["Shard 2"]
    K --> S3["Shard n"]
    S1 --> C1["Redshift"]
    S2 --> C2["DynamoDB"]
    S3 --> C3["S3"]
    S1 --> C4["EMR"]
```

Amazon SNS (Simple Notification Service)

Serviço de mensagens pub/sub da AWS.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    PR1["AWS SDK"] --> SNS["SNS"]
    PR2["AWS CLI"] --> SNS
    PR3["CloudWatch"] --> SNS
    PR4["Serviços AWS"] --> SNS
    SNS --> TOPIC["SNS Topic"]
    TOPIC -->|"filtragem + fanout"| SUB1["Lambda"]
    TOPIC --> SUB2["SQS"]
    TOPIC --> SUB3["Email"]
    TOPIC --> SUB4["HTTP/S"]
```

“Formatos de entrega suportados: texto simples, email, HTTP (webhooks), SMS (mensagens de texto), SQS e Lambda.”

Amazon API Gateway

Programa que se posiciona entre um único ponto de entrada e múltiplos backends — permite throttling, logging, lógica de routing, e formatação de pedidos/respostas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    APP1["App móvel"] --> GW["API Gateway"]
    APP2["App web"] --> GW
    APP3["Dispositivo IoT"] --> GW
    GW --> CACHE["API Gateway Cache"]
    GW --> CW["CloudWatch"]
    GW --> SVC["Serviços AWS<br/>(EC2, Lambda, RDS, ...)"]
```

“Amazon API Gateway: cria APIs seguras no teu ambiente cloud, a qualquer escala.”

Máquinas de Estado e AWS Step Functions

State machine: decide como se passa de um estado para outro, com base numa série de condições — como um fluxograma.

“AWS Step Functions: coordena múltiplos serviços AWS num workflow serverless. Tem uma consola gráfica para visualizar os componentes, aciona e acompanha automaticamente cada passo, faz retry em caso de erro, e regista (log) o estado de cada passo.”

Event Bus e Amazon EventBridge

Event bus: recebe eventos e encaminha-os para um alvo (target), com base em regras.

“Amazon EventBridge (antigo CloudWatch Events): serviço de event bus serverless, usado para integração de aplicações através de streaming de dados em tempo real.”

Resumo: Serviços de Application Integration

Serviço	Tipo	Descrição
SNS	Pub/Sub	Vários formatos de entrega: texto, email, HTTP (webhooks), SMS, SQS, Lambda
SQS	Queueing	Envia eventos para uma fila; outras apps fazem pull da fila para obter mensagens
Step Functions	State machine	Serviço de máquina de estados para orquestrar workflows
EventBridge	Event bus	Event bus serverless
Kinesis	Streaming	Dados em streaming em tempo real, com múltiplos consumidores
Amazon MQ	Message broker	Serviço de message broker gerido, usa Apache ActiveMQ
Managed Kafka (MSK)	Message broker	Apache Kafka totalmente gerido
API Gateway	API management	Criar, publicar, manter, monitorizar e proteger APIs
AppSync	GraphQL	Serviço GraphQL totalmente gerido

AWS Amplify

Framework opinativo e infraestrutura totalmente gerida, para os developers se focarem em construir apps web e mobile.

Componente	Função
Amplify CLI	Linha de comandos para provisionar recursos
Amplify SDK	Biblioteca para integrar a app com os serviços AWS
Amplify UI	Componentes de interface prontos a usar
Amplify Hosting	Hospedagem da aplicação
Amplify Studio	Interface visual para gerir o backend

“Funciona com frameworks JavaScript.”

AWS AppSync

Serviço GraphQL totalmente gerido.

Conceito	Descrição
Schema	Representa o teu modelo de dados
Data sources	De onde os dados são obtidos: DynamoDB, Amazon OpenSearch, AWS Lambda, endpoint HTTP, etc.
Resolvers	Anexados a campos específicos dentro dos teus tipos no schema — implementam as operações de alteração de estado para query, mutation e subscription

- Suporta APIs em tempo real via GraphQL subscriptions.

Tipo de API	Descrição
GraphQL API	Uma única API a partir de múltiplas fontes de dados
Merged API	Coleção de GraphQL APIs que funcionam como uma só — útil quando várias equipas gerem as suas próprias APIs, mas queres tratá-las como uma API única

Opções de caching: nenhuma / caching completo do pedido / caching por resolver (operações ou campos específicos).

■ PARTE 3 — VMs vs. Containers (explicado em detalhe)

O problema das VMs

Numa máquina virtual, cada app corre isolada num SO convidado completo, por cima de um hypervisor. Isto tem um custo:

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph VM["Virtual Machine"]
        OS["SO convidado (ex: Ubuntu)"]
        APP1["Django"]
        APP2["MongoDB"]
        APP3["RabbitMQ"]
        LIB["Libraries, Packages, Binaries"]
        WASTE["■ Espaço desperdiçado"]
    end
    HYP["Hypervisor"] --> VM
    HOST["Host Operating System"] --> HYP
```

- As VMs não fazem o melhor uso do espaço.
- As apps não estão isoladas entre si dentro da mesma VM — podem gerar conflitos de configuração, problemas de segurança ou monopolização de recursos (resource hogging) por uma app em detrimento das outras.

A solução: Containers

Um container empacota a aplicação com tudo o que ela precisa para correr (bibliotecas, dependências), mas partilha o kernel do sistema operativo do host, em vez de ter o seu próprio SO completo.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph C["EC2 Instance com Docker"]
        subgraph CT1["Container 1"]
            D["Django"]
            L1["Lib/Pkg/Bin"]
        end
        subgraph CT2["Container 2"]
            M["MongoDB"]
            L2["Lib/Pkg/Bin"]
        end
        subgraph CT3["Container 3"]
            R["RabbitMQ"]
            L3["Lib/Pkg/Bin"]
        end
        FREE["■ Espaço disponível<br/>para mais containers"]
    end
    DAEMON["Docker Daemon"] --> C
    HOST2["Host Operating System"] --> DAEMON
```

- Os containers permitem correr múltiplas aplicações, virtualmente isoladas umas das outras.

• Podes lançar novos containers e configurar as dependências do SO por container, sem desperdiçar o espaço que sobra, como acontecia com as VMs.

Comparação direta

	Máquina Virtual (VM)	Container
Camada de virtualização	Hypervisor	Container Engine (ex.: Docker Daemon)
O que cada unidade tem	O seu próprio SO completo	Partilha o kernel do SO do host
Peso/arranque	Mais pesado, arranque mais lento (minutos)	Muito mais leve, arranque quase instantâneo (segundos)
Isolamento	Isolamento forte (ao nível do SO)	Isolamento mais leve (ao nível de processo)
Uso de espaço	Tende a desperdiçar recursos reservados e não usados	Aproveita melhor o espaço disponível na mesma instância
Melhor para	Correr sistemas operativos completamente diferentes na mesma máquina física	Empacotar e distribuir aplicações de forma consistente e portátil

"■ Resumo mental: VM = virtualizas o hardware (cada VM pensa que tem o seu próprio computador). Container = virtualizas o sistema operativo (cada container pensa que tem o seu próprio SO, mas todos partilham o mesmo kernel por baixo)."

■ PARTE 4 — Microservices (explicado em detalhe)

Arquitetura Monolítica vs. Microservices

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph MONO["■ Monolithic Architecture<br/>UMA app responsável por tudo<br/>Funcionalidade fortemente acoplada"]
        direction TB
        S1["Services: Load Balancing, Caching, Database"]
        S2["Marketing: HTML/CSS, JavaScript, Assets"]
        S3["Frontend: JS Framework, HTML Templates"]
        S4["Backend: API, ORM, PDFs, Reports, Thumbnails"]
    end
```

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph MICRO["■■ Microservices Architecture<br/>Múltiplas apps, cada uma responsável por UMA coisa<br/>Isoladas e stateles
        direction TB
        M1["Load Balancing"]
        M2["Caching"]
        M3["Database"]
        M4["Queueing"]
        M5["HTML/CSS"]
        M6["JavaScript"]
        M7["Assets"]
        M8["API"]
        M9["ORM"]
        M10["Reports"]
        M11["■■ PDFs (Lambda)"]
        M12["■■ Thumbnails (Lambda)"]
    end
```

	Monolito	Microservices
Nº de aplicações	Uma só, responsável por tudo	Várias, cada uma responsável por uma coisa
Acoplamento	Fortemente acoplado — tudo depende de tudo	Isolado e stateless — cada serviço é independente
Deploy	Deploy de tudo de uma vez, mesmo para uma mudança pequena	Deploy independente de cada serviço

	Monolito	Microservices
Escalar	Escalas a aplicação inteira, mesmo que só uma parte precise	Escalas só o serviço que precisa (ex.: só o serviço de "Thumbnails")
Falha	Uma falha pode derrubar toda a aplicação	Uma falha fica contida nesse serviço específico
Tecnologia	Normalmente uma stack única para tudo	Cada serviço pode usar a tecnologia mais adequada (ex.: PDFs e Thumbnails como funções Lambda)
Complexidade operacional	Mais simples de gerir no início	Mais complexa — precisas de orquestração, service discovery, monitorização distribuída

Vantagens dos microservices: escalabilidade seletiva, resiliência a falhas localizadas, equipas podem trabalhar e fazer deploy independentemente em cada serviço, liberdade tecnológica por serviço.

Desvantagens: maior complexidade de infraestrutura (rede, comunicação entre serviços), mais difícil de testar de ponta a ponta, exige ferramentas de orquestração (é aqui que entra o Kubernetes, a seguir).

■■ PARTE 5 — Kubernetes (K8s)

Sistema de orquestração de containers open-source, para automatizar o deployment, scaling e gestão de containers.

- Criado originalmente pela Google, atualmente mantido pela Cloud Native Computing Foundation (CNCF).
- Chamado habitualmente de "K8s": o "8" representa as letras restantes de "ubernetes".
- Vantagem sobre o Docker sozinho: permite correr containers distribuídos por várias VMs.

Arquitetura do Kubernetes

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    subgraph MASTER["Kubernetes Master"]
        SCHED["Scheduler"]
        CTRL["Controllers"]
        ETCD["Etcd"]
    end
    API["API Server"]
    MASTER --> API

    subgraph NODE1["Node"]
        KL1["Kubelet"]
        PX1["Proxy"]
        subgraph POD1["Pod"]
            C1["Container"]
            C2["Container"]
        end
        subgraph POD2["Pod"]
            C3["Container"]
            C4["Container"]
        end
    end

    subgraph NODE2["Node"]
        KL2["Kubelet"]
        PX2["Proxy"]
        subgraph POD3["Pod"]
            C5["Container"]
            C6["Container"]
        end
        subgraph POD4["Pod"]
            C7["Container"]
            C8["Container"]
        end
    end

    API --> NODE1
    API --> NODE2
```

Componente	Função
Scheduler	Decide em que node cada pod vai correr
Controllers	Garantem que o estado atual do cluster corresponde ao estado desejado
Etcd	Base de dados chave-valor que guarda o estado do cluster
API Server	Ponto de entrada para todos os comandos ao cluster
Node	Uma máquina (VM ou física) que corre pods
Kubelet	Agente que corre em cada node, garante que os containers dos pods estão a correr
Proxy	Gere as regras de rede em cada node
Pod	Grupo de um ou mais containers com storage, recursos de rede e outras configurações partilhadas — é a unidade mais pequena que se pode gerir no Kubernetes

"■ O Kubernetes é ideal para arquiteturas de microservices, onde uma empresa tem dezenas a centenas de serviços para gerir — exatamente o cenário descrito na Parte 4."

■ PARTE 6 — Docker e Podman

Docker

Produto que usa virtualização ao nível do SO para entregar software em pacotes chamados containers.

Componente	Função
Docker CLI	Descarregar, enviar, construir, correr e debugar containers
Dockerfile	Ficheiro de configuração com as instruções para provisionar um container
Docker Compose	Ferramenta + ficheiro de configuração para trabalhar com múltiplos containers
Docker Swarm	Ferramenta de orquestração para gerir arquiteturas de múltiplos containers em produção
Docker Hub	Repositório online público de containers, publicados pela comunidade, para download

Podman

Motor de containers compatível com OCI (Open Container Initiative), pensado como substituto direto ("drop-in replacement") do Docker.

Diferença	Docker	Podman
Arquitetura	Usa um daemon (containerd)	Sem daemon
Suporte a Pods	Não	Sim — tal como o Kubernetes

"O Podman substitui apenas parte do que o Docker faz; costuma usar-se em conjunto com:Buildah — para construir imagens OCISkopeo — para mover imagens de container entre diferentes tipos de storage"

Serviços de Containers na AWS

Categoria	Serviços
Primários (correm os containers)	ECS, AWS Fargate, EKS, AWS Lambda

Categoria	Serviços
Provisioning e deployment	Elastic Beanstalk (EB), App Runner, AWS Copilot CLI
Serviços de suporte	Elastic Container Registry (imagens Docker), X-Ray, Step Functions

■ PARTE 7 — Organizations, Control Tower e Governança

AWS Organizations

Permite criar novas contas AWS e gerir centralmente billing, controlo de acesso, compliance, segurança e partilha de recursos entre contas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    MASTER["■ Master Account<br/>(The Federation)"]
    MASTER --> OU1["■■ Organization Unit:<br/>Starfleet"]
    MASTER --> OU2["■■ Organization Unit:<br/>Federation Planets"]

    OU1 --> A1["Conta: Section 31"]
    OU1 --> A2["Conta: Corps of Engineers"]
    OU1 --> A3["Conta: Starfleet Command"]

    OU2 --> B1["Conta: Earth"]
    OU2 --> B2["Conta: Vulcan"]
    OU2 --> B3["Conta: Betazed"]
```

Conceito	Descrição
Root Account User	Identidade de login única, com acesso completo a todos os serviços e recursos numa conta. Cada conta tem o seu Root Account User
Organization Units (OUs)	Grupos de contas AWS dentro de uma organização — podem conter outras OUs, criando uma hierarquia
Service Control Policies (SCPs)	Dão controlo central sobre as permissões máximas permitidas em todas as contas da organização, ajudando a garantir que as contas se mantêm dentro das regras da organização

"■■ O AWS Organizations, depois de ativado, não pode ser desativado. Podes criar quantas contas AWS quiseres — uma delas será sempre a conta Master/Root da organização."

AWS Control Tower

Ajuda empresas a montar rapidamente um ambiente multi-conta AWS seguro, fornecendo um ambiente de base (baseline) para começar com uma arquitetura multi-conta.

Componente	Descrição
Landing Zone	Ambiente de base: ativa o AWS SSO, logging centralizado via AWS CloudTrail, e auditoria de segurança entre contas
Account Factory	Automatiza o provisionamento de novas contas na organização, padronizando o processo
Guardrails	Regras de governança pré-empacotadas, para segurança, operações e compliance, que podes aplicar a toda a empresa ou a grupos específicos de contas

AWS Config

"Change management: ter um processo formal para monitorizar, aplicar e corrigir alterações. Compliance as Code: usar programação para automatizar a monitorização, aplicação e correção de alterações."

AWS Config: framework de compliance as code, que permite gerir alterações numa base por região.

Quando usar o AWS Config?

- Queres que um recurso se mantenha configurado de uma forma específica, por motivos de compliance.
- Queres acompanhar alterações de configuração ao longo do tempo.
- Precisas de uma lista de todos os recursos existentes.
- Queres analisar potenciais fragilidades de segurança.

Como funciona

Defines regras para detetar situações específicas, e um plano de remediação — se a regra for violada, o Config pode corrigir automaticamente (ou notificar alguém para o fazer).

Exemplo — IAM Password Policy Conformance Pack:

- Comprimento mínimo da password
- Exigir letras minúsculas
- Exigir números
- Idade máxima da password
- Exigir prevenção de reutilização
- Exigir letras maiúsculas
- Exigir símbolos

Tagging (Etiquetagem)

Uma tag é um par chave-valor que atribuis a recursos AWS.

Exemplos de tags:

Key	Value
dept	it
status	approved
project	enterprise
location	london

Porque organizar recursos com tags

- Gestão de recursos
- Gestão e otimização de custos
- Gestão de operações
- Segurança
- Governança e compliance regulatório
- Automação
- Otimização de workloads

Resource Groups

Coleção de recursos que partilham uma ou mais tags. Aparecem no cabeçalho global da consola AWS, e também em Systems Manager.

■■ PARTE 8 — Cloud Adoption Framework (CAF)

Whitepaper da AWS para ajudar a migrar de on-premise para a AWS.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    CAF["AWS Cloud Adoption Framework"] --> B["Business"]
    CAF --> P["Platform"]
    CAF --> PE["People"]
    CAF --> S["Security"]
    CAF --> G["Governance"]
    CAF --> O["Operations"]
```

6 áreas de foco: Business, Platform, People, Security, Governance, Operations.

■ PARTE 9 — Serviços Orientados ao Negócio

Serviço	Função
Amazon Connect	Serviço de call center virtual
Amazon WorkSpaces	Serviço de ambiente de trabalho remoto virtual
Amazon WorkDocs	Serviço de colaboração partilhada (semelhante ao SharePoint)
Amazon Chime	Serviço de videoconferência (semelhante ao Zoom)
Amazon WorkMail	Serviço gerido de email empresarial, contactos e calendário
Amazon Pinpoint	Gestão de campanhas de marketing — envio de emails/SMS direcionados
Amazon SES (Simple Email Service)	Serviço de email transacional, integrável nas tuas aplicações
Amazon QuickSight	Serviço de Business Intelligence (BI)

■■ PARTE 10 — Serviços de Provisioning

“Provisioning: alocação ou criação de recursos e serviços para um cliente.”

Serviço	Função
Elastic Beanstalk	PaaS para fazer deploy de aplicações web facilmente
AWS OpsWorks	Serviço de configuration management, com instâncias geridas do Chef e Puppet (open-source)
AWS CloudFormation	Serviço de modelação e provisionamento de infraestrutura — automatiza o provisionamento ao escrever templates JSON/YAML (Infrastructure as Code)
AWS Quick Start	Pacotes pré-construídos para lançar e configurar rapidamente compute, rede, storage e outros serviços
AWS Marketplace	Catálogo digital de milhares de listagens de software de fornecedores independentes
AWS Amplify	Framework mobile e web que provisiona múltiplos serviços AWS como backend, com foco em serverless (DynamoDB, API Gateway, ...)
AWS App Runner	Facilita o deploy rápido de aplicações web e APIs containerizadas, a escala, sem experiência prévia de infraestrutura

Serviço	Função
AWS Copilot	CLI que permite lançar e gerir facilmente aplicações containerizadas na AWS
AWS CodeStar	Interface unificada para gerir atividades de desenvolvimento de software num único sítio
AWS CDK (Cloud Development Kit)	Ferramenta de IaC que permite usar a tua linguagem de programação preferida, gerando templates CloudFormation como mecanismo de IaC

■ Progresso: 9h45min de vídeo assistido.

Aula 11

Aula 11 — PaaS, Serverless, Windows, Logging, ML/AI e Big Data

Aula 11 — PaaS, Serverless, Windows, Logging, ML/AI e Big Data

■■ PARTE 1 — AWS Elastic Beanstalk

Platform as a Service (PaaS): permite aos clientes desenvolver, correr e gerir aplicações sem terem de construir e manter toda a complexidade de infraestrutura normalmente associada ao desenvolvimento e lançamento de apps.

“Elastic Beanstalk: PaaS para fazer deploy de aplicações web com pouco ou nenhum conhecimento da infraestrutura subjacente — foca-te em escrever código, em vez de pipelines de deployment e tarefas de DevOps.”

- ■■ Não é recomendado para apps de produção de grandes empresas (falta de controlo fino sobre a infraestrutura).
- É alimentado por templates CloudFormation, que configuram automaticamente por ti:Elastic Load BalancerAuto Scaling GroupsBase de dados RDSInstâncias EC2 pré-configuradasAmbiente Docker, se preferires correr containerizado

“Deploy de apps muito simples e rápido.”

■ PARTE 2 — Serverless

O servidor subjacente, a infraestrutura e o SO são geridos pelo cloud provider. Por norma, serverless é altamente disponível, escalável e económico — pagas pelo que usas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["Scale to Zero<br/>sem uso = sem custo"]
    B["Pay for Value<br/>não pagas por servidores parados"]
```

Princípio	Significado
Scale to Zero	Se os recursos não estão a ser usados, o serverless não custa nada
Pay for Value	Não pagas por servidores "idle" (parados à espera)

Serviços serverless

Serviço	Descrição
DynamoDB	Base de dados NoSQL key/value e de documentos, serverless. Desenhada para escalar a milhares de milhões de registos — não geres shards
S3	Serviço de object storage serverless. Uploads de ficheiros muito grandes e em quantidade ilimitada. Pagas pelo que armazenas
ECS Fargate	Orquestração de containers serverless — igual ao ECS, mas pagas on-demand por container em execução (com ECS "puro" tens sempre um servidor EC2 ligado, mesmo sem containers a correr)
AWS Lambda	Serviço de funções serverless — corres código sem provisionar nem gerir servidores. Carregas um pequeno pedaço de código, escolhes a memória e o tempo máximo de execução. Cobrado com base no tempo de execução

Serviço	Descrição
Step Functions	Serviço de máquina de estados — coordena múltiplos serviços AWS em workflows serverless. Facilita partilhar dados entre Lambdas, ter um grupo de Lambdas à espera umas das outras, passos lógicos. Também funciona com tasks do Fargate
Aurora Serverless	Versão on-demand e serverless do Aurora. Tem a maioria dos benefícios do Aurora, mas pode ter cold starts, ou é ideal quando não tens muito tráfego constante

■ PARTE 3 — Windows na AWS

Serviço	Descrição
Windows Server on EC2	Instâncias EC2 a correr Windows Server
SQL Server on RDS	SQL Server como base de dados gerida
AWS Directory Service	Corre o Microsoft Active Directory (AD) como serviço gerido
AWS License Manager	Facilita gerir licenças de software de fornecedores como a Microsoft
Amazon FSx for Windows File Server	Storage gerido e escalável para Windows
AWS SDK	Escreves código na tua linguagem preferida para interagir com a API da AWS. Suporta .NET — favorito dos developers Windows
Amazon WorkSpaces	Corre um ambiente de trabalho virtual — podes lançar um desktop Windows 10 para fornecer postos de trabalho seguros e duráveis
AWS Lambda	Suporta PowerShell
AWS Migration Acceleration Program	Metodologia de migração para mover grandes empresas Windows para a AWS

AWS License Manager em detalhe

“BYOL (Bring Your Own License): reutilizar uma licença de software já existente para correr software de um fornecedor num serviço de computação cloud. Poupa dinheiro, já que a licença pode ter sido comprada em massa ou com desconto, mais barato do que comprar de novo.”

AWS License Manager: serviço que gere as tuas licenças de software.

- Gere software licenciado com base em: vCPUs, cores físicos, sockets, ou número de máquinas.
- Produtos de fornecedores como: Microsoft, IBM, SAP, Oracle.

“■■ Para licenças de Windows Server e SQL Server, normalmente precisas de usar um Dedicated Host (liga ao conceito já visto na Aula 7).”

■ PARTE 4 — Serviços de Logging

AWS CloudTrail

Regista chamadas à API entre serviços AWS. Permite governança, compliance, auditoria operacional e de risco da tua conta.

Perguntas que o CloudTrail responde:

- Quem criou este bucket?
- Quem lançou aquela instância EC2 cara?

- Detetar agentes maliciosos
- Automatizar respostas
- Detetar má configuração por parte de developers

Os 4 "W" de cada evento

	Informação
Where	IP de origem
When	Hora do evento
Who	Utilizador, user agent
What	Região, recurso, ação

“Os logs ficam disponíveis durante os últimos 90 dias, via Event History. Se precisares de mais do que 90 dias, tens de criar um Trail.”

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    CT["CloudTrail"] -->|Trail| S3["■ S3<br/>(logs guardados)"]
    S3 --> ATH["Amazon Athena<br/>(interface SQL/GUI<br/>para consultar os logs)"]
```

Amazon CloudWatch

Coleciona dados de múltiplos serviços.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    CW["Amazon CloudWatch"] --> LOGS["■ Logs<br/>local centralizado para<br/>logs de app e logs cloud"]
    CW --> MET["■ Metrics<br/>séries temporais de dados"]
    CW --> EVT["■ Events (EventBridge)<br/>trigger por condição"]
    CW --> ALM["■ Alarms<br/>notificações por threshold"]
    CW --> DASH["■ Dashboards<br/>visualizações"]
    CW --> INS["■ Logs Insights<br/>pesquisa interativa"]
```

Componente	Descrição
CloudWatch Logs	Local centralizado para guardar dados de log da cloud e das tuas apps
CloudWatch Metrics	Conjunto ordenado por tempo de pontos de dados — uma variável monitorizada ao longo do tempo. Métricas predefinidas para EC2: uso de CPU, operações de leitura/escrita em disco, tráfego de rede de entrada/saída, etc.
CloudWatch Events (EventBridge)	Aciona ações com base numa condição (ex.: a cada hora, tirar um snapshot do servidor)
CloudWatch Alarms	Notificações baseadas em métricas. Defines um threshold; quando violado, o alarme muda de estado (OK, Alarm, ou Insufficient Data — acabou de começar, não disponível, ou dados insuficientes). Ao mudar de estado, podes acionar: notificações, Auto Scaling Group, ação na EC2
CloudWatch Dashboard	Cria dashboards de visualização com base em métricas
CloudWatch Logs Insights	Pesquisa e analisa interativamente os teus dados de log: filtragem robusta, menos trabalhoso do que exportar para S3 e analisar com Athena, suporta todos os tipos de log, usa sintaxe de query própria. ■■ As queries expiram ao fim de 15 minutos, e os resultados ficam disponíveis durante 7 dias

AWS X-Ray

Sistema de tracing distribuído: vês como os dados se movem de uma app para outra, quanto tempo demorou, e se falhou em algum ponto do percurso.

■ PARTE 5 — Machine Learning e IA

A hierarquia AI → ML → Deep Learning

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TD
    AI["■ Artificial Intelligence<br/>máquinas que imitam comportamento humano"]
    AI --> ML["■ Machine Learning<br/>máquinas que melhoram numa tarefa<br/>sem programação explícita"]
    ML --> DL["■■ Deep Learning<br/>redes neuronais artificiais inspiradas<br/>no cérebro humano, para problemas complexos"]
```

Termo	Definição
Artificial Intelligence (AI)	Máquinas que executam tarefas que imitam comportamento humano
Machine Learning (ML)	Máquinas que melhoram numa tarefa sem programação explícita
Deep Learning	Usa redes neuronais artificiais, inspiradas no cérebro humano, para resolver problemas complexos

Amazon SageMaker

Serviço totalmente gerido para construir, treinar e fazer deploy de modelos de machine learning.

Framework suportado	Descrição
Apache MXNet on AWS	Framework de deep learning
TensorFlow	Biblioteca de inteligência de máquina
PyTorch	Framework de machine learning

Serviço relacionado	Função
SageMaker Ground Truth	Serviço de etiquetagem (labeling) de dados, usado para treinar modelos
Amazon Augmented AI	Serviço de revisão com intervenção humana

Serviços de IA especializados

Serviço	Função
Amazon CodeGuru	Análise de código com ML — revê e sugere melhorias à qualidade do código
Amazon Lex	Interface de conversação (chatbots por voz e texto)
Amazon Personalize	Recomendações em tempo real
Amazon Polly	Texto para voz (text-to-speech)
Amazon Rekognition	Reconhecimento de imagem e vídeo — deteta e etiqueta objetos, pessoas, celebridades, etc.
Amazon Transcribe	Voz para texto (speech-to-text)
Amazon Textract	OCR — extrai texto de documentos digitalizados
Amazon Translate	Tradução automática via redes neuronais
Amazon Comprehend	Processamento de linguagem natural (NLP) — encontra relações no texto para gerar insights
Amazon Forecast	Previsão de séries temporais
AWS Deep Learning AMI	EC2 pré-instalada com frameworks populares de deep learning (TensorFlow, PyTorch, ...)
AWS Deep Learning Containers	Imagens Docker pré-instaladas com os mesmos frameworks

Serviço	Função
AWS DeepComposer	Teclado musical com machine learning
AWS DeepLens	Câmara de vídeo com deep learning
AWS DeepRacer	Carrinho de corrida autónomo com machine learning
Amazon Elastic Inference	Anexa aceleração GPU de baixo custo a instâncias EC2, reduzindo até 75% o custo de correr inferência de deep learning
Amazon Fraud Detector	Serviço de deteção de fraude totalmente gerido
Amazon Kendra	Motor de pesquisa empresarial com ML — usa linguagem natural para sugerir respostas a perguntas, em vez de simples correspondência de palavras-chave

IA — serviços mais recentes

Serviço	Função
Amazon Bedrock	Serviço cloud de LLM (Large Language Model) — gera respostas em texto e imagem. Pensa nisto como um "ChatGPT" da AWS
Amazon CodeWhisperer	Assistente de código com IA — equivalente ao GitHub Copilot
Amazon DevOps Guru	Usa ML para analisar dados operacionais, métricas e eventos da app, detetando anomalias operacionais
Amazon Lookout (Equipment/Metrics/Vision)	Usa modelos de ML para controlo de qualidade e inspeções automatizadas
Amazon Monitron	Prevê downtime não planeado de equipamento
AWS Neuron	Corre workloads de deep learning em instâncias baseadas em AWS Inferentia e AWS Trainium

Generative AI

Tipo de IA capaz de gerar conteúdo novo: texto, imagens, música ou outras formas de media.

■ PARTE 6 — Big Data e Analytics

“Big Data: volumes massivos de dados estruturados/não estruturados, tão grandes que são difíceis de mover e processar.”

Serviços de Big Data

Serviço	Função
Amazon Athena	Serviço de query interativo serverless. Pega em ficheiros CSV/JSON num bucket S3 e carrega-os numa tabela SQL temporária, para correres queries. Usa quando queres fazer query a ficheiros CSV ou JSON
Amazon CloudSearch	Serviço de pesquisa full-text — para adicionares pesquisa ao teu website
Amazon Elasticsearch Service	Cluster Elasticsearch gerido
Kinesis Data Streams	Serviço de streaming de dados em tempo real — crias produtores que enviam dados para a stream; múltiplos consumidores podem consumir os dados
Kinesis Firehose	Versão serverless e mais simples do Data Streams — pagas on-demand pela quantidade de dados consumida
Amazon Kinesis Data Analytics	Corre queries em tempo real contra dados na stream, podes criar relatórios
Amazon Kinesis Video Streams	Permite analisar e processar vídeo em streaming, em tempo real

Serviço	Função
Managed Kafka Service (MSK)	Apache Kafka totalmente gerido
Amazon Redshift	Data warehouse à escala de petabytes — para gerar analytics/relatórios rapidamente a partir de grandes volumes de dados
Amazon QuickSight	Dashboard de Business Intelligence — cria dashboards de negócio para apoiar decisões (semelhante ao Power BI)
AWS Data Pipeline	Automatiza o movimento de dados
AWS Glue	Serviço de ETL (Extract, Transform, Load)
AWS Lake Formation	Repositório centralizado, organizado e protegido, que guarda todos os teus dados

Amazon QuickSight em detalhe

Dashboard de Business Intelligence que permite ingerir dados de storage ou bases de dados AWS, para visualizar rapidamente dados de negócio, com pouco conhecimento de programação ou fórmulas.

- Usa o SPICE (motor de cálculo in-memory) para performance ultra-rápida, a escala.
- QuickSight ML Insights: deteta anomalias e faz previsões (forecasting) precisas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    S1["S3 / RDS / Redshift / ..."] --> QS["Amazon QuickSight"]
    QS --> SPICE["■ SPICE<br/>motor in-memory"]
    SPICE --> DASH["■ Dashboards<br/>de negócio"]
    SPICE --> ML["■ ML Insights<br/>anomalias + forecasting"]
```

■ Progresso: 10h54min de vídeo assistido.

Aula 12

Aula 12 — Frameworks de ML, Hardware, Well-Architected Framework e Migração

Aula 12 — Frameworks de ML, Hardware, Well-Architected Framework e Migração

■ PARTE 1 — Frameworks e Ferramentas de ML/DL

Framework	Descrição
Apache MXNet	Adotado pela AWS, suporta programação imperativa e simbólica
PyTorch	Biblioteca de tensores otimizada para deep learning, usando GPU e CPU
TensorFlow	Framework de machine learning de baixo nível
Keras	Framework de alto nível, construído por cima do TensorFlow e distribuído com ele
Apache Spark	Motor de analytics unificado para processamento de dados a larga escala
SparkML	Conjunto uniforme de APIs de alto nível, para ajudar a criar e afinar pipelines práticos de machine learning

Apache MXNet em detalhe

Framework de deep learning/machine learning adotado pela AWS, que suporta programação imperativa e simbólica, em múltiplas linguagens: Python, Java, Julia, MATLAB, JavaScript, etc.

Característica	Descrição
Escalável	Desenhado para infraestrutura cloud distribuída
Flexível	Suporta programação imperativa e simbólica
Portátil	Pode ser usado em dispositivos de baixa capacidade, edge devices, e em compute serverless
Multi-linguagem	Suporta várias linguagens de programação

Duas interfaces

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    MX["Apache MXNet"] --> G["Gluon API<br/>programação imperativa"]
    MX --> M["Module API<br/>programação simbólica"]
```

■■ PARTE 2 — Hardware para Machine Learning

Intel vs. ARM

	Intel	ARM
Tipo	Fabricante de semicondutores, instruction set x86	Instruction set com menos instruções
Eficiência energética	—	Normalmente melhor eficiência energética, logo custos mais baixos

Processadores especializados

Processador	Função
Intel Xeon Scalable	CPU de alta performance, desenhado para empresas — comummente usado em instâncias AWS
Habana Gaudi	Processador de treino de IA, feito à medida para treinar modelos de deep learning

GPU (Graphics Processing Unit)

Processador especializado para renderizar rapidamente imagens e vídeo de alta resolução em simultâneo. Pode ter milhares de núcleos, otimizados para muitas tarefas em paralelo.

Boa para:

- Renderização de gráficos
- Mineração de criptomoeda
- Deep learning e Machine Learning

CUDA e NVIDIA

- NVIDIA: fabricante de GPUs.
- CUDA: plataforma de computação paralela e API da NVIDIA, que permite aos developers usar GPUs com suporte CUDA.

Instâncias EC2 com GPU

Instância	GPU
EC2 P3	Tesla V100
EC2 G3	Tesla M60
EC2 G4	T4
EC2 P4	Tesla A100

“Integra-se com o NVIDIA Deep Learning SDK.”

■ PARTE 3 — Amazon Athena (em detalhe)

Serviço de query interativo que facilita analisar dados diretamente a partir do S3 — baseado em Apache Presto.

Pode fazer duas coisas

Modo	Descrição
Athena SQL	Corre queries SQL em buckets S3
Apache Spark on Amazon Athena	Corre analytics de dados interativamente

“Serverless — pagas pelo que usas.”

Integra-se com: CloudFormation, CloudFront, CloudTrail, DataZone, ELB, EMR, AWS Glue Data Catalog, IAM, QuickSight, S3 Inventory, Step Functions, Systems Manager Inventory, VPC.

AWS Glue

Serviço de integração de dados serverless.

“Cria, corre e monitoriza visualmente pipelines de ETL (Extract, Transform, Load), para carregar dados nos teus data lakes.”

■■ PARTE 4 — AWS Well-Architected Framework

O AWS Well-Architected Framework é um whitepaper criado pela AWS para ajudar os clientes a construir usando as boas práticas definidas pela AWS. Está disponível em aws.amazon.com/architecture/well-architected.

O framework está dividido em 5 secções, chamadas pilares, cada uma a abordar um aspeto diferente (uma "lente") que pode ser aplicada a um workload na cloud.

Estrutura geral

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    WL["■■ O Teu Workload na Cloud"]
    FW["■■ AWS Well-Architected Framework<br/>• General Definitions<br/>• General Design Principles<br/>• The Review Process"]
    WL --- FW
    FW --- P1["Operational<br/>Excellence"]
    FW --- P2["Security"]
    FW --- P3["Reliability"]
    FW --- P4["Performance<br/>Efficiency"]
    FW --- P5["Cost<br/>Optimization"]
```

Camada	O que representa
Your Cloud Workload	A aplicação/sistema que estás a construir na cloud
Well-Architected Framework	A "fundação" sobre a qual assenta o workload — inclui três componentes: General Definitions, General Design Principles e The Review Process
5 Pilares	As colunas que sustentam o framework, cada uma focada num aspeto diferente

“■■ A analogia do print: "If the foundation is not solid, structural problems will arise." — tal como num edifício, se a fundação (a arquitetura) não for sólida, vão surgir problemas estruturais no teu workload mais tarde.”

Os 5 pilares

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    VAL["■ Business Value"]
    P1["Operational Excellence<br/>correr e monitorizar sistemas"]
    P2["Security<br/>proteger dados, mitigar riscos"]
    P3["Reliability<br/>mitigar e recuperar de disrupções"]
    P4["Performance Efficiency<br/>usar recursos de computação eficazmente"]
    P5["Cost Optimization<br/>obter o menor preço"]
    P1 --> VAL
    P2 --> VAL
    P3 --> VAL
    P4 --> VAL
    P5 --> VAL
```

Pilar	Foco
Operational Excellence	Correr e monitorizar sistemas
Security	Proteger dados, mitigar risco

Pilar	Foco
Reliability	Mitigar e recuperar de disrupções
Performance Efficiency	Usar os recursos de computação de forma eficaz
Cost Optimization	Obter o menor preço possível

Definições gerais

Termo	Definição
Component	Código, configurações, um recurso AWS
Workload	Conjunto de componentes que trabalham em conjunto
Milestones	Mudanças-chave na tua arquitetura ao longo do ciclo de vida do produto
Architecture	Como os componentes trabalham em conjunto num workload
Technology Portfolio	Coleção de workloads necessários para o negócio funcionar

Anatomia de um pilar

Cada pilar do Well-Architected Framework é estruturado como:

Secção	Descrição
Design Principles	Lista de princípios de design a considerar na implementação (a parte mais importante para o exame)
Definition	Visão geral da boa prática
Best Practices	Informação sobre cada boa prática
Resources	Documentação de apoio

Pilar: Operational Excellence

Princípio	Descrição
Perform operations as code	Limita o erro humano e permite respostas consistentes → IaC
Make frequent, small, reversible changes	Rollbacks, mudanças incrementais, blue/green, CI/CD
Refine operations procedures frequently	Usa game days para simular tráfego ou falhas nos teus workloads de produção
Anticipate failure	Prepara-te antecipadamente para falhas
Learn from operational failures	Aprende com falhas operacionais passadas

Pilar: Security

Princípio	Descrição
Implement strong identity foundation	Menor privilégio, identidade centralizada, evitar credenciais de longa duração
Enable traceability	Monitoriza alertas e audita ações/mudanças no ambiente; recolha de logs e métricas; automatiza investigação e remediação
Apply security at all layers	Segurança em todas as camadas
Automate security best practices	Automatiza as boas práticas de segurança
Protect data in transit and at rest	Protege os dados tanto em trânsito como em repouso
Keep people away from data	Minimiza o acesso humano direto aos dados
Prepare for security events	Ferramentas para detetar, investigar e recuperar de incidentes

Pilar: Reliability

- Recuperar automaticamente de falhas
- Testar procedimentos de recuperação
- Escalar horizontalmente para aumentar a disponibilidade agregada do sistema
- Parar de adivinhar a capacidade
- Gerir mudanças por automação (IaC)

Pilar: Performance Efficiency

- Democratizar tecnologias avançadas
- Ir global em minutos
- Usar arquiteturas serverless
- Experimentar mais frequentemente
- Considerar a "simpatia mecânica" (mechanical sympathy — desenhar software que aproveita bem as características do hardware subjacente)

Pilar: Cost Optimization

- Implementar gestão financeira cloud
- Adotar o modelo de consumo
- Medir a eficiência geral
- Parar de gastar dinheiro em trabalho pesado não diferenciado (undifferentiated heavy lifting)
- Analisar e atribuir despesa

AWS Well-Architected Tool

Ferramenta de auditoria para verificar o quão alinhado está o teu workload com o Well-Architected Framework — essencialmente uma checklist, com referências próximas, que ajuda a montar um relatório para partilhar com executivos e stakeholders-chave.

AWS Architecture Center

Portal web com boas práticas e arquiteturas de referência.

■ PARTE 5 — Organização e Princípios

Arquitetura: On-Premise vs. AWS

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph ONPREM["■ Empresa On-Premise"]
        direction TB
        T1["Equipa Centralizada:<br/>Technical Architect<br/>Solution Architect<br/>Data Architect<br/>Networking Architect<br/>
        T2["Gerida por TOGAF<br/>ou Zachman Framework"]
    end
    subgraph AWSORG["■■ AWS"]
        direction TB
        A1["Equipas Distribuídas"]
```

```
    A2["Practices: raise the bar"]
    A3["Mechanisms: verificações<br/>automatizadas de standards"]
    A4["Amazon Leadership Principles"]
    A5["Comunidade virtual de SMEs<br/>e Principal Engineers<br/>(ex: lunchtime talks)"]
end
```

	On-Premise	AWS
Estrutura da equipa	Centralizada (Technical/Solution/Data/Networking/Security Architect)	Distribuída
Suporte	TOGAF ou Zachman Framework	Practices, Mechanisms (verificações automatizadas), Amazon Leadership Principles
Partilha de conhecimento	—	Comunidade virtual de SMEs e Principal Engineers (ex.: lunchtime talks, reaproveitados como material de onboarding)

Amazon Leadership Principles

Conjunto de princípios usados na tomada de decisão, resolução de problemas, brainstorming e contratação:

- Customer Obsession
- Ownership
- Invent and Simplify
- Are Right, A Lot
- Learn and Be Curious
- Hire and Develop the Best
- Insist on the Highest Standards
- Think Big
- Bias for Action
- Frugality
- Earn Trust
- Dive Deep
- Have Backbone; Disagree and Commit
- Deliver Results
- Strive to be Earth's Best Employer
- Success and Scale Bring Broad Responsibility

Princípios gerais de design

- Parar de adivinhar as necessidades de capacidade
- Testar sistemas à escala de produção
- Automatizar para facilitar a experimentação arquitetural
- Permitir arquiteturas evolutivas
- Orientar a arquitetura com base em dados
- Melhorar através de game days

■ PARTE 6 — Custos e Migração

Total Cost of Ownership (TCO)

Estimativa financeira destinada a ajudar compradores e proprietários a determinar os custos diretos e indiretos de um produto ou serviço. Muito útil ao migrar de on-premise para a cloud.

Exemplo: licenças de software

On-Premise (CapEx)	AWS (OpEx)
Taxas de licença de software	Taxas de subscrição
Implementação	Implementação
Configuração	Configuração
Formação	Formação
Segurança física	—
Hardware	—
Pessoal de TI	—
Manutenção	—

CapEx vs. OpEx

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    subgraph CAPEX["■ CapEx (Capital Expenditure)"]
        C1["Gasto adiantado em<br/>infraestrutura física"]
        C2["Servidores, storage, rede,<br/>backup, datacenter, pessoal técnico"]
        C3["Tens de adivinhar<br/>o gasto à partida"]
    end
    subgraph OPEX["■ OpEx (Operational Expenditure)"]
        O1["Custos não-físicos"]
        O2["Leasing de software e<br/>personalização de funcionalidades"]
        O3["Formação, suporte cloud,<br/>faturação por consumo"]
        O4["Podes experimentar um produto<br/>sem investir em equipamento"]
    end
```

	CapEx	OpEx
Natureza	Gasto adiantado em infraestrutura física	Custos não-físicos
Exemplos	Servidores, storage, rede, backup/arquivo, datacenter, pessoal técnico	Leasing de software, formação, suporte cloud, faturação por uso de compute/storage
Planeamento	Tens de adivinhar o gasto antecipadamente	Podes experimentar um produto/serviço sem investir em equipamento

Mudança de pessoal de TI

Ao migrar para a cloud:

- A empresa continua a precisar de uma equipa de TI durante a fase de migração.
- As funções transitam para novos papéis cloud — ex.: de "networking" para "cloud networking".
- O foco passa de gerir infraestrutura para gerar receita.

AWS Pricing Calculator

Ferramenta gratuita de estimativa de custos, usada no browser, sem precisar de conta AWS.

AWS Migration Evaluator

Ferramenta de estimativa usada para determinar o custo da tua infraestrutura on-premise existente, para comparar com o custo AWS de uma migração cloud planeada.

“Usa um agentless collector, que recolhe dados da tua infraestrutura on-premise sem precisar de instalar agentes.”

EC2 VM Import/Export

Importa imagens de máquinas virtuais para a EC2. Suporta: VMware, Citrix, Hyper-V, etc.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["1. Preparar a imagem<br/>virtual para upload"] --> B["2. Upload para S3"]
    B --> C["3. AWS CLI importa a imagem<br/>e gera uma AMI"]
    C --> D["4. AMI pronta a usar no EC2"]
```

AWS Database Migration Service (DMS)

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    SRC["■■ Base de dados origem"] --> SEP["Source Endpoint"]
    SEP --> TASK["Replication Task"]
    TASK --> TEP["Target Endpoint"]
    TEP --> TGT["■■ Base de dados destino"]
```

Suporta: Oracle DB, Microsoft SQL, MongoDB, MySQL, PostgreSQL.

■ Progresso: 11h47min de vídeo assistido.

Aula 13

Aula 13 — Serviços Gratuitos, Planos de Suporte, Billing, Trusted Advisor, SLA e Custos

Aula 13 — Serviços Gratuitos, Planos de Suporte, Billing, Trusted Advisor, SLA e Custos

■ PARTE 1 — Serviços Gratuitos

Serviço	Descrição
IAM	Gestão de identidades e acessos
VPC	Rede virtual privada
Auto Scaling	Escalar recursos automaticamente
CloudFormation	Infraestrutura como código
Elastic Beanstalk	Deploy de aplicações geridas
OpsWorks	Gestão de configuração (Chef/Puppet)
Amplify	Desenvolvimento de apps web/mobile
AppSync	APIs GraphQL geridas
CodeStar	Gestão de projetos de desenvolvimento
Organizations + Consolidated Billing	Gestão de múltiplas contas e faturação única
AWS Cost Explorer	Análise de custos e utilização

“■■ Estes serviços são gratuitos por si, mas provisionam outros serviços que podem ter custo (aplica-se de Auto Scaling até CodeStar).”

■ PARTE 2 — Support Plans (importante)

	Basic	Developer	Business	Enterprise
Preço	0 USD/mês	29 USD/mês	100 USD/mês	15 000 USD/mês
Suporte	Email só para billing e conta	Email técnico (24h até resposta)	Email + chat/telefone 24/7	Email + chat/telefone 24/7
Suporte de terceiros	—	■ Não	■	■
General guidance	—	< 24h	< 24h	< 24h
System impaired	—	< 12h	< 12h	< 12h
Production system impaired	—	—	< 4h	< 4h
Production system down	—	—	< 1h	< 1h
Business-critical system down	—	—	—	< 15 min
Trusted Advisor	7 checks	7 checks	Todos os checks	Todos os checks
Extras Enterprise	—	—	—	Personal concierge + TAM

“■ Nota pessoal: "print costs" — falta colar o print da tabela de custos.”

TAM (Technical Account Manager)

Orientação proativa e suporte reativo para ajudar a ter sucesso na AWS — apenas disponível no plano Enterprise.

- Constrói soluções e dá orientação técnica
- Garante que os serviços AWS se mantêm operacionais, reduzindo custo e complexidade
- Compreende as necessidades do negócio e desenvolve uma relação de confiança
- Colabora com a equipa de negócio profissional
- Faz revisões de disrupções de serviço, métricas, etc.

Support Center

- Só a root account pode criar um case
- Também é onde se pode mudar o plano de suporte

■ PARTE 3 — AWS Marketplace e Consolidated Billing

AWS Marketplace

- Milhares de listagens de software de outros fornecedores
- Produtos podem ser gratuitos ou ter custo associado
- Também podes vender as tuas soluções a outros clientes AWS

Consolidated Billing

Permite pagar várias contas AWS com uma única fatura.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    M["■ Master Account<br/>(Cost Explorer)"]
    A1["Member Account"] --> M
    A2["Member Account"] --> M
```

Descontos por volume

“Quanto mais usas, mais poupas.”

Exemplo — Data Transfer:

Escalão	Preço
Primeiros 10 TB	0,17 USD/GB
Próximos 40 TB	0,13 USD/GB

“Com consolidated billing, o uso de todas as contas soma-se, por isso atinges escalões mais baratos mais depressa.”

■ PARTE 4 — AWS Trusted Advisor

Ferramenta de recomendações que monitoriza automaticamente a tua conta AWS e dá recomendações acionáveis. É uma checklist automatizada de boas práticas.

5 categorias: Cost Optimization · Performance · Security · Fault Tolerance · Service Limits

Basic | Developer — 7 checks

Os 6 primeiros são de segurança:

#	Check
1	MFA no Root
2	Security Groups — portas específicas sem restrição
3	Permissões S3
4	EBS public snapshot
5	RDS public snapshot
6	IAM use — desencorajar o uso do acesso root
7	Service Limits (todos os checks de service limits são gratuitos)

Business | Enterprise — todos os checks

Categoria	Exemplos
Cost Optimization	Load balancers inativos, Elastic IPs não associados
Performance	Instâncias EC2 com alta utilização
Security	MFA no root, rotação de access keys IAM
Fault Tolerance	Backups RDS
Service Limits	VPC

■ PARTE 5 — Service Level Agreements (SLA)

SLA = compromisso formal sobre o nível esperado de serviço.

Termo	Significado	Exemplo
SLI (Service Level Indicator)	Métrica/medição que indica o desempenho que o cliente está a receber num dado momento	Uptime, performance, disponibilidade, latência, taxa de erro
SLO (Service Level Objective)	Objetivo que o fornecedor acordou cumprir, com um alvo percentual específico	Disponibilidade de 99,99% num período de 3 meses

Exemplos de SLA

Serviço	SLA
DynamoDB	99,99% disponível, com uptime mensal por região AWS
Compute (EC2, EBS, ECS, Fargate)	Nível região (deploy em vários AZs/regiões) e nível instância (governa cada instância EC2 individualmente)
RDS	Instâncias Multi-AZ com uptime mensal de pelo menos 99,95%

“■ Se algum SLA não for cumprido, tens direito a receber service credits.”

Dashboards de saúde

Dashboard	Função
Service Health Dashboard	Mostra o estado geral dos serviços AWS (ícone + detalhes do estado)
Personal Health Dashboard	Alertas e orientação sobre eventos AWS que podem afetar o teu ambiente

■ PARTE 6 — AWS Abuse

A equipa AWS Trust and Safety lida com abusos originados na plataforma AWS:

Tipo de abuso	Descrição
Spam	Emails não solicitados vindos da AWS
Port scanning	Logs mostram IPs da AWS a enviar pacotes para várias portas do teu servidor
Denial of Service (DoS)	Ataques de negação de serviço
Intrusion attempts	IPs AWS usados para tentar entrar nos teus recursos
Hosting prohibited content	Alojamento de conteúdo proibido
Distributing malware	Recursos AWS usados para distribuir software criado para causar dano

"■ Não uses o AWS Support para isto → envia email para abuse@amazonaws.com ou preenche o AWS Abuse Report Form."

■ PARTE 7 — Free Tier, Credits e APN

AWS Free Tier

Permite usar a AWS sem custo:

- Primeiros 12 meses (limites mensais)
- Alguns serviços têm uso gratuito para sempre até um certo limite mensal

Serviço	Free Tier
EC2	t2.micro, 750 h/mês durante 1 ano
RDS	db.t2.micro, 750 h/mês durante 1 ano
ELB	750 h/mês durante 1 ano
CloudFront	50 GB de data transfer
Amazon Connect	90 min de tempo de chamada

AWS Credits

- Equivalem a USD na plataforma AWS
- Podem ser obtidos por: AWS Activate Startup Program, participação em surveys, etc.
- Têm data de expiração
- Podem ser usados na maioria dos serviços AWS

AWS Partner Network (APN)

Programa global de parceiros da AWS: formação e marketing.

Tipo de parceiro	Função
Consulting Partner	Ajuda empresas a usar a AWS
Technology Partner	Constrói tecnologia por cima da AWS, como serviço

- Níveis (tiers): Select → Advanced → Premier, com custos diferentes
- É preciso ter certificações AWS para ser parceiro
- Parceiros podem obter AWS credits

■ PARTE 8 — Ferramentas de Gestão de Custos

AWS Budgets

Alertas quando excedes ou estás a aproximar-te do orçamento definido.

- Tipos: cost, usage e reservation budgets
- Acompanhamento mensal, trimestral ou anual
- Alertas suportam: EC2, RDS, Redshift, ElastiCache
- Primeiros 2 budgets gratuitos, depois 0,02 USD/dia
- Budget Reports: relatórios por email (diário, semanal ou mensal) com a performance do budget

AWS Cost and Usage Reports (CUR)

Folha de cálculo detalhada para analisar os custos AWS.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    CUR["■ Cost & Usage Report"] --> S3["■ S3"]
    S3 --> ATH["■ Athena<br/>(base de dados consultável)"]
    ATH --> QS["■ QuickSight<br/>(gráficos de billing)"]
```

Cost Allocation Tags

Metadados opcionais anexados a recursos AWS, que permitem gerar um Cost and Usage Report por tag.

Billing Alarms

Alarmes CloudWatch para monitorizar o gasto.

1. Ativar billing alerts
2. Ir a CloudWatch → Alarms
3. Escolher Billing como métrica
4. Especificar métrica e condições

AWS Cost Explorer

Visualizar, compreender e gerir custos e utilização AWS ao longo do tempo.

- Especificar tipo, intervalo e agregação
- Os relatórios por defeito ajudam a obter insights rapidamente

■ Progresso: 12h39min de vídeo assistido.

Aula 14

Aula 14 — Pricing API, Segurança, Encriptação, Compliance e Serviços de Segurança AWS

Aula 14 — Pricing API, Segurança, Encriptação, Compliance e Serviços de Segurança AWS

■ PARTE 1 — Pricing API e Savings Plans

AWS Pricing API

API	Formato
Query API	Via JSON
Batch (Bulk) API	Ficheiros em lote (nas notas: "via html")

Savings Plans — follow along

Cost Explorer → Savings Plans → escolher o plano (Compute | EC2 | SageMaker) → anos de compromisso → ...

TAM (Technical Account Manager)

(Repetido da Aula 13) Orientação proativa e suporte reativo, apenas no plano Enterprise: constrói soluções, mantém serviços operacionais reduzindo custo e complexidade, e faz revisões de disrupções e métricas.

■■ PARTE 2 — Fundamentos de Segurança

Defense in Depth (7 camadas)

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart TB
    L1["■■ Data — encriptação dos dados"]
    L2["■ Application — apps seguras, sem vulnerabilidades"]
    L3["■■ Compute — acesso a VMs (portas, on-premise, cloud)"]
    L4["■ Network — segmentação e controlo de acesso"]
    L5["■ Perimeter — proteção DDoS"]
    L6["■ Identity & Access — controlo de acesso e change control"]
    L7["■ Physical — acesso ao datacenter"]
    L7 --> L6 --> L5 --> L4 --> L3 --> L2 --> L1
```

Camada	Foco
Data	Usar encriptação nos dados
Application	Aplicações seguras e livres de vulnerabilidades
Compute	Acesso às VMs (portas, on-premise, cloud)
Network	Limitar comunicação entre recursos com segmentação e controlo de acesso
Perimeter	Proteção DDoS para filtrar ataques de larga escala antes de causarem negação de serviço
Identity and Access	Controlar o acesso à infraestrutura e o change control

Camada	Foco
Physical	Limitar o acesso ao datacenter

CIA Triad

Modelo que descreve a base dos princípios de segurança e a relação de trade-off entre eles. Mencionado numa publicação do NIST de 1977.

Pilar	Descrição
Confidentiality	Proteger os dados de quem não está autorizado. Na prática: chaves criptográficas para encriptar dados, e chaves para encriptar chaves (envelope encryption)
Integrity	Manter e garantir a exatidão e completude dos dados durante todo o ciclo de vida
Availability	A informação tem de estar disponível quando é necessária

Vulnerabilidades

Fraqueza na aplicação (falha de design ou bug de implementação) que permite a atacantes causar dano aos stakeholders.

Exemplos: buffer overflow, validação de dados imprópria, memory leak, violação do menor privilégio, uso de algoritmo criptográfico quebrado ou arriscado.

■ PARTE 3 — Encriptação, Hashing e Assinaturas Digitais

Conceitos de encriptação

Termo	Definição
Cryptography	Técnicas para comunicação segura na presença de terceiros (adversaries)
Encryption	Codificar informação com uma chave e uma cipher, guardando dados sensíveis num formato ininteligível. Recebe plaintext e produz ciphertext
Cipher	Algoritmo que faz a encriptação ou desencriptação
Ciphertext	Resultado da encriptação do plaintext
Cryptographic key	Variável usada com o algoritmo para encriptar/desencriptar

Simétrica vs. Assimétrica

	Simétrica	Assimétrica
Chaves	A mesma chave para codificar e descodificar	2 chaves: uma para codificar, outra para descodificar
Exemplo	AES (Advanced Encryption Standard)	RSA (Rivest–Shamir–Adleman)

Hashing e Salting

Função que aceita um valor de tamanho arbitrário e o mapeia para uma estrutura de tamanho fixo.

- Processo one-way (não reversível)
- Determinística: o mesmo input dá sempre o mesmo output

- Usado para guardar passwords na base de dados: o utilizador introduz a password, faz-se o hash e compara-se com o hash guardado
- Algoritmos: MD5, SHA-256, bcrypt

“■■ Se um atacante souber a função usada e roubar a base de dados, pode enumerar um dicionário de passwords para descobrir a original.”

Salting: string aleatória, desconhecida do atacante, que entra na função de hash para mitigar a natureza determinística.

Assinaturas Digitais (Digital Signature)

Esquema matemático para verificar a autenticidade de mensagens ou ficheiros. Dá evidência de adulteração (tamper evidence):

- Alguém modificou os dados?
- Os dados não vêm do remetente esperado?

Algoritmo	Função
Key generation	Gera a chave pública e privada
Signing	Gera a assinatura com a chave privada + a mensagem
Signature verification	Verifica a autenticidade com a chave pública

“■ ssh-keygen é o comando conhecido para gerar um par de chaves pública/privada.”

Encriptação In Transit vs. At Rest

	In Transit	At Rest
Quando	Dados a mover-se entre localizações	Dados guardados em storage
Tecnologias	TLS/SSL (TLS 1.2 e 1.3 são a boa prática)	AES, RSA

■ PARTE 4 — Compliance e Penetration Testing

Compliance Programs

Políticas e procedimentos internos de uma empresa para cumprir leis, regras e regulamentos.

Standard	Descrição
ISO (International Organization for Standardization)	27001 — guia de implementação de controlos; 27017 — foco reforçado em segurança cloud
SOC (System and Organization Controls)	SOC 1, SOC 2 (controlos internos, políticas e procedimentos relacionados com a segurança de um sistema), SOC 3
PCI DSS	Payment Card Industry Data Security Standard

Penetration Testing

Ciberataque simulado e autorizado a um sistema, para avaliar a sua segurança.

■ Permitido (ex.)	■ Proibido
EC2, NAT Gateways, Load Balancers, RDS, CloudFront, API Gateways, ...	DNS zone walking via Route 53 hosted zones; simulação de DDoS

“■ É preciso enviar um formulário AWS a avisar que vais fazer pentesting.”

AWS Artifact

Relatórios de compliance da AWS (PDFs).

■ PARTE 5 — Serviços de Segurança AWS

Amazon Inspector

Hardening: eliminar o máximo de riscos de segurança possível.

Corre um benchmark de segurança contra EC2 específicas.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    A["1. Instalar o AWS agent na EC2"] --> B["2. Correr um assessment ao target"]
    B --> C["3. Rever findings e remediar"]
```

“Benchmark popular: CIS — 699 checks.”

DDoS e AWS Shield

DDoS: tentativa maliciosa de interromper o tráfego normal inundando um site com grandes quantidades de tráfego falso.

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    ATT["■ Atacante"] --> VMS["Múltiplas VMs<br/>enviam tráfego em simultâneo"]
    VMS --> APP["Aplicação / Serviço"]
    APP --> CRASH["■ Crash / serviço parado"]
```

AWS Shield: serviço gerido de proteção DDoS para workloads a correr na AWS.

AWS GuardDuty

IDS/IPS (Intrusion Detection / Prevention System): dispositivo ou software que monitoriza redes ou sistemas à procura de atividade maliciosa ou violações de política.

GuardDuty: serviço de threat detection que monitoriza continuamente atividade suspeita e comportamento não autorizado, usando machine learning para analisar os logs AWS.

AWS VPN

Estabelece um túnel seguro e privado entre a tua rede/dispositivo e a rede global AWS.

Tipo	Liga
Site-to-Site VPN	Rede on-premise → VPC
Client VPN	Utilizadores → AWS ou redes on-premise

“IPSec: conjunto de protocolos de rede que autentica e encripta os pacotes de dados para comunicação segura.”

AWS WAF

Protege a tua web app de exploits web comuns.

- Escreves as tuas próprias regras para permitir ou negar tráfego
- Rulesets de parceiros de segurança AWS no AWS WAF Rules Marketplace
- Usa-se com CloudFront ou Application Load Balancer
- Cobre o OWASP Top 10: injection, broken authentication, sensitive data exposure, XML external entities, broken access control, security misconfigurations, cross-site scripting, insecure deserialization, componentes com vulnerabilidades conhecidas, logging e monitoring insuficientes
- Pode fazer bypass de endereços IP

Hardware Security Module (HSM)

Desenhado para guardar chaves de encriptação, mantendo-as em memória e nunca as escrevendo em disco.

	Multi-tenant	Single-tenant
Serviço AWS	AWS KMS	AWS CloudHSM
Compliance	FIPS 140-2 Level 2	FIPS 140-2 Level 3

AWS KMS

Serviço gerido para criar e controlar as chaves de encriptação (1 USD por chave).

- Multi-tenant
- Usa envelope encryption

Diagrama / fluxo

```
Diagrama Mermaid (código-fonte):
flowchart LR
    D["■ Data"] -->|encriptada com| DK["■ Data Key"]
    DK -->|encriptada com| MK["■■ Master Key"]
    MK --> KMS["■■ Guardada no KMS"]
```

AWS CloudHSM

HSM single-tenant como serviço, que automatiza provisionamento de hardware, patching de software, alta disponibilidade e backups.

- Permite gerar e usar as tuas chaves em FIPS 140-2 Level 3
- É o mais caro, porque recebes uma peça de hardware dedicada

AWS Security Hub

CSPM (Cloud Security Posture Management): gera um security score para determinar a tua postura de segurança.

- Podes ativar standards (coleções de controlos de segurança → são AWS Config rules)

AWS Audit Manager

Audita continuamente o uso da AWS para simplificar a avaliação de risco e compliance.

- Framework library: suporta vários standards (SOC 2, HIPAA, ...)

- Podes criar assessments, rever a evidência recolhida e gerar um relatório
- Recolhe evidência continuamente com AWS Config e AWS Security Hub

AWS Firewall Manager

Configura e gere regras de firewall de forma centralizada:

- AWS WAF e WAF Classic
- Security Groups
- AWS Shield Advanced
- Amazon Route 53 Resolver DNS Firewall
- ...

■■ PARTE 6 — Comparações de Serviços

AWS Config vs. AWS AppConfig

	AWS Config	AWS AppConfig
Função	Ferramenta de governance — compliance as code	Automatiza o deploy de configurações de aplicações
Como	Cria regras que verificam se os recursos estão configurados como esperado; se não, notifica ou faz auto-remediation	Escreves um validator para garantir que a variável alterada não parte a web app; monitoriza deployments e faz rollback em caso de erro

SNS vs. SQS

	SNS (Simple Notification Service)	SQS (Simple Queue Service)
Modelo	Passa mensagens (pub/sub)	Fila de mensagens, entrega garantida
Funcionamento	Envia notificações a subscribers de tópicos (geralmente emails de texto simples); pode repetir envios falhados	Coloca mensagens numa fila; as apps fazem pull com o AWS SDK
Extras	—	Retém mensagens até 14 dias; ordem sequencial ou paralela; garante entrega pelo menos uma vez

SNS vs. SES vs. Pinpoint vs. WorkMail

Todos enviam emails.

Serviço	Uso
SNS	Notificações pub/sub, geralmente emails de texto simples
SES (Simple Email Service)	Emails transacionais, disparados por ações na app (signup, reset de password, faturas) → emails HTML
Pinpoint	Emails promocionais: campanhas, segmentação de contactos, customer journeys
WorkMail	Cliente de email web, semelhante a Gmail/Outlook — emails da empresa

Amazon Inspector vs. AWS Trusted Advisor

Ambas são ferramentas de segurança que fazem auditorias.

	Amazon Inspector	Trusted Advisor
Âmbito	Audita uma única EC2 selecionada	Visão holística de recomendações em vários serviços e boas práticas
Output	Relatório de uma longa lista de checks (ex.: 699)	Não gera PDF
Exemplos	Resultados do benchmark de segurança	"Tens portas abertas nestes security groups", "deves ativar MFA na root account"

AWS Artifact vs. Amazon Inspector

Ambos produzem PDFs.

	AWS Artifact	Amazon Inspector
Pergunta	"Porque devo confiar na AWS?"	"Como sei que a minha EC2 é segura? Prova-o."
Output	Relatório de segurança baseado em frameworks globais de compliance (SOC, PCI)	Audita uma EC2 e gera relatório com uma longa lista de checks (ex.: 699)

Serviços "Connect"

Serviço	Descrição
Direct Connect	Fibra ótica dedicada do teu datacenter para a AWS. Se precisares de ligação segura → AWS VPN por cima do Direct Connect
Amazon Connect	Call center como serviço
MediaConvert	Transcodifica vídeos para diferentes formatos (ex.: 1000 vídeos para converter, aplicar watermarks, ou inserir um vídeo de introdução no início de cada um)

“■ Nas tuas notas escreveste "media connect: new version of elastic transcoder". O serviço que substitui o Elastic Transcoder é o AWS Elemental MediaConvert; o MediaConnect é outro serviço (transporte de vídeo ao vivo).”

Elastic Transcoder vs. MediaConvert

Ambos transcodificam vídeo.

Serviço	Descrição
Elastic Transcoder	A forma antiga
MediaConvert	Serviço de transcoding mais robusto

Load Balancers: ELB — ALB vs. NLB vs. GWLB vs. CLB

O Elastic Load Balancer tem 4 tipos:

Tipo	Camada	Características
ALB (Application)	Layer 7 — HTTP/S	Regras de routing com base em informação HTTP; pode usar AWS WAF
NLB (Network)	Layer 3 e 4 — TCP e UDP	Performance extrema para tráfego TCP e TLS; milhões de pedidos por segundo, latências ultra baixas
GWLB (Gateway)	—	Deploy de uma frota de virtual appliances de terceiros que suportam GENEVE
CLB (Classic)	Layer 3, 4 e 7	Para apps construídas na rede EC2-Classic; não usa target groups

■ Progresso: 12h39min de vídeo assistido.